

# DeckerPlus: Whole Program Attack Surface Reduction via Compiler Analysis and Transformations

ALEXANDRA HUSSAR, Georgia Institute of Technology, United States

SHARJEEL KHAN, Georgia Institute of Technology, United States

CHRISTOPHER PORTER, Georgia Institute of Technology, United States

SANTOSH PANDE, Georgia Institute of Technology, United States

Modern code reuse attacks take full advantage of bloated software. Attackers piece together short sequences of instructions in otherwise benign code to carry out malicious actions. Mitigating these reusable code snippets, known as gadgets, has become one of the prime focuses of attack surface reduction research. While some debloating techniques remove parts of software that contain such gadgets, other methods focus on making them unusable by breaking up chains of them, thereby substantially diminishing the possibility of code reuse attacks. Due to a high number of vulnerabilities, third-party libraries (such as `glibc`) used to be the main focus of this research but recently the scope of such attacks has enlarged to whole programs and thus, the techniques have emerged that deal with whole applications. Attack surface reduction efforts have typically tried to eliminate such attacks by subsetting (debloating) the application, e.g., via user-specified inputs, configurations. However, such techniques suffer from limitations in soundness, i.e., the software might crash during normal executions on valid inputs, or they may be conservative and leave a large amount of attack surface untackled.

In this work, we present a general, whole-program attack surface reduction technique called `DECKERPLUS` that significantly reduces gadgets that are accessible to an attacker during an execution phase (called a deck) and has minor performance degradation. `DECKERPLUS` requires no user inputs and leaves all features intact. It uses static analysis to determine key function sets that should be enabled/disabled at runtime. The runtime system enables these function sets at the specified program points during execution. We implement two extensions to improve the baseline technique utilizing function cloning and inlining. As a result, our baseline framework achieves 70.3% average total gadget reduction on SPEC CPU 2017, 88.5% on GNU coreutils, and 89.0% across the application workloads (`nginx`, `Redis`, `lighttpd`, and `xpdf`). Function cloning improves these reductions to 75.5%, 89.6%, and 94.2%, respectively, and inlining improves them further to 77.6%, 94.7%, and 97.8%. Overall, the baseline and extensions achieve these reductions with modest runtime overhead: average slowdown is 6.5% on SPEC CPU 2017, `nginx` averages 1.062 $\times$  slowdown with baseline `DECKERPLUS` and 1.011 $\times$  with inlining, `Redis` throughput is essentially unchanged, `lighttpd` remains close to baseline throughput, and GNU coreutils overhead is negligible. In addition, we provide a gadget chain-breaking case study, including detailed JOP gadget metrics on both Linux and Windows, and show that our framework breaks the shell-spawning chain in all cases.

CCS Concepts: • **Security and privacy** → **Software security engineering**; • **Software and its engineering** → **Software verification and validation**.

---

Authors' Contact Information: Alexandra Hussar (corresponding author), College of Computing, Georgia Institute of Technology, Atlanta, Georgia, United States; e-mail: [ahussar3@gatech.edu](mailto:ahussar3@gatech.edu); Sharjeel Khan, College of Computing, Georgia Institute of Technology, Atlanta, Georgia, United States; e-mail: [smkhan@gatech.edu](mailto:smkhan@gatech.edu); Christopher Porter, College of Computing, Georgia Institute of Technology, Atlanta, Georgia, United States; e-mail: [cporter35@gatech.edu](mailto:cporter35@gatech.edu); Santosh Pande, College of Computing, Georgia Institute of Technology, Atlanta, Georgia, United States; e-mail: [santosh.pande@cc.gatech.edu](mailto:santosh.pande@cc.gatech.edu).



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2471-2566/2026/11-ART40

<https://doi.org/10.1145/3833877>

Additional Key Words and Phrases: Attack surface reduction, software debloating, code reuse attacks, program analysis, program security

#### ACM Reference Format:

Alexandra Hussar, Sharjeel Khan, Christopher Porter, and Santosh Pande. 2026. DeckerPlus: Whole Program Attack Surface Reduction via Compiler Analysis and Transformations. *ACM Trans. Priv. Sec.* 29, 4, Article 40 (November 2026), 35 pages. <https://doi.org/10.1145/3833877>

## 1 Introduction

Attack surface reduction has become increasingly important as software continues to include large amounts of unused code. A recent investigation found that applications in a typical Ubuntu desktop installation leave an average of 95% of GNU libc code unused [43]. Such code can still contain vulnerabilities or provide gadgets for code reuse attacks.

Memory-safety vulnerabilities in C and C++ remain a major source of exploitable bugs, accounting for approximately 70% of security bugs reported by Microsoft and Chrome [13, 14]. Although memory-safe languages such as Rust offer a path forward [31], existing C and C++ software will remain widely deployed. Therefore, improving the security of current C and C++ applications remains necessary.

In this work, we reduce the executable code and reusable gadgets exposed to an attacker during program execution. Listing 1 motivates the problem with a shell-spawning **return-oriented programming (ROP)** chain.

```
gdgt 1 addr  -> pop rax; ret;          // load "/bin/sh" address into rax
'/bin/sh'
gdgt 2 addr  -> pop rcx; ret;          // load .bss address into rcx
bss addr     -> 0x00800000
gdgt 3 addr  -> mov qword ptr [rcx],rax;ret; // write "/bin/sh" to .bss
gdgt 4 addr  -> pop rax; ret;          // load null terminator into rax
0x00000000
gdgt 5 addr  -> pop rcx; ret;          // load .bss+8 address into rcx
bss addr + 8 -> 0x00800008
gdgt 6 addr  -> mov qword ptr [rcx],rax;ret; // write null to .bss+8
gdgt 7 addr  -> pop rdi; ret;          // set execve arg1: pointer to "/bin/sh"
bss addr     -> 0x00800000
gdgt 8 addr  -> pop rsi; ret;          // set execve arg2: pointer to null
bss addr + 8 -> 0x00800008
gdgt 9 addr  -> pop rdx; ret;          // set execve arg3: pointer to null
bss addr + 8 -> 0x00800008
gdgt 10 addr -> pop rax; ret;          // load execve syscall number (0x3b)
0x0000003b
gdgt 11 addr -> syscall;              // invoke execve("/bin/sh", null, null)
```

Listing 1. An ROP gadget chain from nginx that spawns a shell with execve.

Ropper automatically generated this chain from the nginx executable [37, 47]. It writes `/bin/sh` into memory, prepares the arguments to `execve`, and invokes the system call. We assume the attacker has already exploited a memory vulnerability and can corrupt control data to initiate the chain. This assumption is realistic for applications such as nginx; for example, control-flow bending has been used to exploit CVE-2013-2028 and obtain arbitrary writes [11].

Effective code reuse attacks do not require a Turing-complete gadget set. A short chain that invokes a system call, changes memory permissions, or leaks sensitive data may be sufficient.

Moreover, the `nginx` chain in Listing 1 includes gadgets beginning at unintended instruction offsets. Such gadgets are possible because `x86` uses variable-length instruction encoding, increasing the number of reusable instruction sequences available to an attacker.

Attack surface reduction attempts to restrict these sequences by removing or disabling code. However, two challenges remain. First, even small binaries may contain enough gadgets to construct useful attacks. Second, some debloating techniques rely on incomplete workloads or user inputs and may therefore remove required functionality, violating program semantics [7]. We define a **sound** transformation as one that preserves the original program semantics. This work asks whether the dynamically available attack surface can be reduced enough to break useful gadget chains while preserving soundness during benign execution.

We propose **DECKERPLUS**, a sound whole-application attack surface reduction technique that dynamically enables and disables groups of functions. This article makes the following contributions:

- (1) The first sound, whole-application technique for on-demand loading and purging for attack surface reduction with low runtime overhead.
- (2) Function cloning and inlining extensions that improve attack surface reduction and can also reduce runtime overhead.
- (3) An evaluation on SPEC CPU 2017, GNU coreutils, and standalone applications including `nginx`, `Redis`, `lighttpd`, and `xpdf`, showing gadget reduction comparable to unsound techniques.
- (4) Evidence that **DECKERPLUS** can break short but highly damaging gadget chains.

## 2 Background and Motivation

Code injection preceded modern code reuse attacks. Early attacks injected code into writable memory and executed it, motivating **data execution prevention (DEP)** [2]. DEP enforces the write XOR execute ( $W \oplus X$ ) property, preventing a page from being both writable and executable. Attackers subsequently developed code reuse techniques. A classic return-to-libc attack redirects compromised control data to existing GNU libc functions [36, 56]. For example, an attacker who controls the arguments to `mprotect` can make a writable page executable and re-enable code injection. **Address space layout randomization (ASLR)** [39] makes such targets more difficult to locate, although multiple bypass techniques have been demonstrated [19, 50].

More advanced techniques include ROP [49], **jump-oriented programming (JOP)** [4], and **call-oriented programming (COP)** [46]. These attacks chain short instruction sequences, or gadgets, already present in executable code. Although gadgets can support Turing-complete computation, useful attacks often require much less, such as leaking a secret or spawning a shell. Existing defenses have mitigated some forms of code reuse, but attackers have also developed bypasses, and protection against non-return-based gadgets remains less mature (Section 7). Attack surface reduction instead limits the code and gadgets available to an attacker.

A central challenge is handling **may-use code**. Such code may be unnecessary during some executions but required under particular inputs or runtime conditions. Permanently removing it can break valid executions, while leaving it available at all times increases the exposed attack surface. Existing attack surface reduction techniques address this tradeoff in different ways, with varying consequences for soundness, precision, and applicability. We focus on four representative systems that motivate our work: Piece-wise Compiler [43], Chisel [22], Razor [41], and BlankIt [40].

Piece-wise Compiler removes library functions proven unreachable from the complete application. It is sound and requires no user-supplied workloads, but its conservative static analysis must retain may-use code, limiting attack surface reduction. It also requires specially compiled libraries and has not been demonstrated on glibc. Chisel uses reinforcement learning and test inputs to construct a reduced application. It removes may-use code when that code is not exercised by the supplied tests,

Table 1. Broad Properties of Four State-of-the-Art Debloating Techniques for Security: Piece-Wise Compiler, Chisel, Razor, and Blankit

|                          | PW | Chsl | Rzr | BI |
|--------------------------|----|------|-----|----|
| Works on application     |    | ✓    | ✓   |    |
| Works on library         | ✓  |      | ✓   | ✓  |
| Works on binary          |    |      | ✓   | ✓  |
| No user input needed     | ✓  |      |     | ✓  |
| No training needed       | ✓  |      | ✓   |    |
| Is sound                 | ✓  |      |     | ✓  |
| Can debloat may-use code |    |      |     | ✓  |

which can eliminate functionality needed under other valid inputs and cause crashes or incorrect output. It therefore requires source code and a user specification of expected behavior.

Razor uses heuristics and test inputs to debloat application binaries. Like Chisel, it removes may-use code based on observed executions and may therefore eliminate functionality required under untested conditions. BlankIt instead handles may-use code dynamically by loading library functions before use and purging them afterward. Its runtime predictor reduces the number of functions exposed at each library callsite. However, BlankIt targets libraries rather than complete applications, and its prediction-based function sets cannot provide the same guarantees as conservative static analysis.

Table 1 summarizes these systems. The most important distinctions are soundness and the ability to reduce may-use code. Static techniques preserve soundness by retaining may-use code, but this leaves more attack surface available. Profile- and test-driven techniques can remove may-use code more aggressively, but incomplete workloads may cause them to eliminate functionality required under other valid executions. Dynamic techniques can adapt to runtime contexts and expose may-use code only when needed, but prediction introduces additional correctness and security concerns.

Machine-learning-based systems also depend on representative training data and must handle mispredictions. In Chisel, an incorrect compile-time decision can remove may-use code that is required under unobserved valid executions. BlankIt predicts at runtime, where the system must respond to prediction failures and protect predictor inputs from memory corruption. Chisel and Razor additionally require test cases that adequately represent when may-use code is needed.

To the best of our knowledge, no general technique simultaneously provides sound whole-application attack surface reduction, dynamic handling of may-use code, and no requirement for training data or user-supplied workloads. Existing approaches either retain may-use code to preserve soundness, handle it only within libraries, or remove it from complete applications at the risk of breaking valid executions. These limitations motivate DECKERPLUS.

### 3 Overview

#### 3.1 Proposed Solution

We propose DECKERPLUS, a compiler and runtime system for whole-application attack surface reduction. It is sound, requires no user input or hardware changes, and combines static analysis with runtime page-permission management. DECKERPLUS follows the principle that only code needed at the current execution phase should be executable. It groups functions that may execute from a given program point into *decks*, while making all other code inaccessible. Because precisely determining the next executed functions at runtime would be too expensive, particularly inside

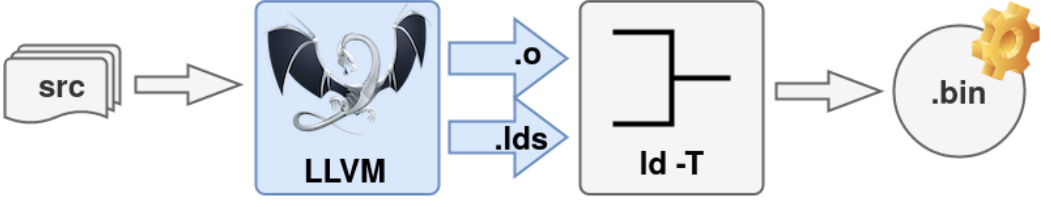


Fig. 1. High-level view of the compiler part. The LLVM pass outputs a custom linker script and instrumented object file.

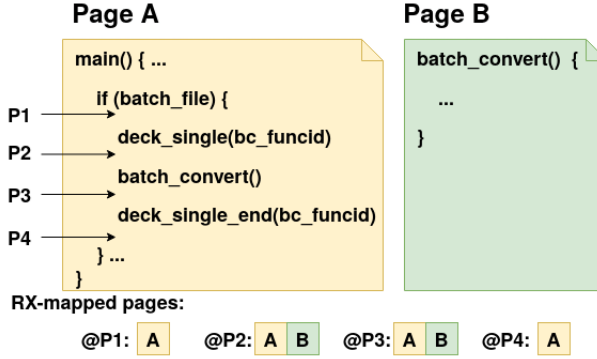


Fig. 2. High-level runtime example from GNU coreutils' date. The set of RX-mapped pages increases at P2 to include a called function.

loops, DECKERPLUS uses conservative static analysis to compute tight overapproximations of the functions that may execute within each region.

At selected program points, DECKERPLUS enables the corresponding deck before execution enters the region and disables it afterward. Instrumenting every call site would incur excessive overhead, especially inside loops, so decking is placed at points such as outermost loop entrances and calls that begin longer interprocedural regions. Each deck conservatively includes all functions reachable before the next decking point, including possible indirect-call targets, thereby preserving soundness. The implementation operates at page granularity: creating and tearing down a deck marks its code pages **read-execute (RX)** and **read-only (RO)**, respectively.

Figure 1 shows the compiler workflow. Source code is processed by an LLVM pass that identifies where decks must be enabled and disabled, inserts the corresponding runtime calls, and emits an instrumented object file. The pass also partitions functions to improve attack surface reduction and generates a custom linker script. The linker consumes the object file and script through its standard `-T` option to produce the final binary; the linker itself is unmodified.

Figure 2 illustrates runtime decking using GNU coreutils' date program [15]. When invoked with the `-f` option, `main` calls `batch_convert`. For simplicity, the example shows two code pages: page A contains `main`, and page B contains `batch_convert`. Before the call, the compiler-inserted `deck_single` operation uses the compile-time identifier `bc_funcid` to map page B RX. After `batch_convert` returns, `deck_single_end` maps page B back to RO.

Program points P1–P4 show the executable-page set throughout this sequence. At P1, only page A is executable. Page B is enabled at P2 before control enters `batch_convert`, remains executable while the function runs at P3, and is disabled again at P4 after the return. Thus, gadgets on page

B are exposed only while that function may execute. More generally, finer-grained decks reduce the amount of executable code and the number of gadgets available to an attacker at any given point.

### 3.2 Soundness

Guaranteeing soundness is a key contribution of DECKERPLUS. We define a sound transformation as one that preserves program semantics. A straightforward sound design would instrument every interprocedural control-flow transfer, enabling all pages required by the callee before entry and disabling them afterward. Because this would be prohibitively slow, DECKERPLUS removes redundant instrumentation using conservative callgraph reachability and loop analysis while still enabling every function that may execute in a region before control enters it. Section 4.1.1 describes this process using encompassed functions.

This guarantee depends on the compiler callgraph conservatively representing every possible interprocedural transfer. A missing edge could exclude a valid callee from its deck, causing benign execution to reach a page that was not enabled. Inline assembly can hide such transfers from LLVM. We observed no missing edges in our Linux or Windows benchmarks; a production implementation can warn or fail when inline assembly contains unresolved control flow and can detect some omissions by identifying functions unexpectedly unreachable from main.

### 3.3 Threat Model

We trust the operating system, compiler, loader, and DECKERPLUS runtime, while application code and third-party libraries may contain vulnerabilities. We focus on C and C++ programs and exclude dynamically generated and self-modifying code. Our experiments target 64-bit applications, although the decking mechanism is architecture-agnostic and can also apply to 32-bit targets. Because DECKERPLUS controls executable permissions at page granularity according to interprocedural control flow, we expect a similar proportional reduction in available pages across these targets. We assume the attacker can exploit a memory vulnerability, corrupt control data, and initiate a gadget chain. DECKERPLUS does not prevent the initial corruption or protect indirect branch targets; instead, it reduces the executable surface available after exploitation by incrementally enabling code along the program's expected interprocedural control flow.

The DECKERPLUS runtime library and its metadata must be protected from modification and control-flow diversion, following a similar assumption in BlankIt [40]. This protection can be implemented using in-process isolation [26], hardware segmentation, or software fault isolation [27]. Together with the trusted loader, it prevents attackers from invoking the runtime arbitrarily or modifying the statically computed function identifiers and deck metadata. Arguments to the runtime API described in Section 4.2 are statically determined and passed in registers, so they cannot normally be altered by the attacker. Indirect calls are the exception because their target addresses are determined at runtime. Protecting the integrity of those targets is an orthogonal problem addressed by mechanisms such as **Control-flow integrity (CFI)** and **Control pointer integrity (CPI)** (Section 7) and is outside the scope of DECKERPLUS.

Under these assumptions, a gadget chain requiring code from multiple disabled decks cannot complete because its gadgets are not simultaneously executable. A control transfer to a gadget on an RO page raises a runtime exception. Instrumented deck-enabling calls execute only with their corresponding teardown operations, as described in Section 4.

## 4 Framework

The framework consists of a compiler and runtime component. The compiler is responsible for inserting calls in the application to the runtime to create and tear down decks. The runtime receives



requests from the application and enables and disables the code pages associated with each deck. In this section, we discuss each part separately and then cover optimizations.

#### 4.1 Compiler Component

DECKERPLUS's compiler component is an LLVM [28] pass that can be divided further into two parts: instrumentation and linker script output. During instrumentation, DECKERPLUS identifies function calls and loops and instruments them appropriately with calls to the runtime. As the pass does this, it collects critical static information for organizing the text section, which it uses to create a custom linker script. A key idea of this work is to keep the decks as lean as possible. We define a deck as a group of functions enabled at a program point by turning their page permissions to RX. For the best security, each deck could consist of one function. Such a scheme would incur high overheads, especially for call chains that execute inside loops, making the scheme untenable. Thus, based on the context surrounding a program point, static analysis identifies which functions should be part of a deck at a given program point and inserts calls to the runtime accordingly.

*4.1.1 Analyzing for Decks.* Analysis and instrumentation for decks is heavily organized around loops. Loops are problematic because adding code for enabling or disabling a deck inside them can cause significant performance degradation. We define two terms: **encompassed** and **non-encompassed** functions to distinguish between two kinds of a function's static loop context. A function is encompassed if it is called inside a loop or is reachable through the callgraph by some function called within a loop through a caller-callee relation. To determine the encompassed function set, the pass first identifies all functions called within a loop. Then, it takes the transitive closure of any functions reachable from that set using the caller-callee relation shown by the callgraph. The non-encompassed function set is the set of all functions minus the encompassed function set.

DECKERPLUS's default treatment of loops is to err on the side of performance. Therefore, it tries to avoid adding decks inside of loops because if this deck instrumentation is invoked in a surrounding intra- or interprocedural loop at runtime, the instrumentation will incur repeated invocations, leading to high overheads. Interprocedurally this implies that DECKERPLUS cannot instrument inside of encompassed functions, either. This raises a problem for loop-enclosed indirect calls, whose static target set can be large and whose precise dynamic value is often unknown until execution is inside the loop. Note that depending on its loop-calling context, a given function might be encompassed and non-encompassed. The edge-based placement of RX-RO tackles such cases.

To handle these different cases, the pass instruments four different types of decks that will invoke the runtime: (1) Single, (2) Loop, (3) Reachable, and (4) Indirect. The **single** deck is used when a non-encompassed function calls a non-encompassed function. Because the callee is known to be non-encompassed (i.e., not part of some transitive closure that lies within a loop), only a single function needs to be mapped RX (i.e., the callee itself). The **loop** deck is placed at the outermost loop header for any loop nest in any non-encompassed function. It is designed to map all functions that can be reached interprocedurally within that loop. The **reachable** deck is placed in a non-encompassed function before any calls to encompassed functions.

The **indirect** deck is for function pointers. The challenge of function pointers is that their exact targets are often not known statically. Therefore, the compiler cannot precisely determine what needs to be mapped to RX until runtime. Function pointer analysis can help narrow the possible targets but would still be an overapproximated set at compile time (which would limit attack surface reduction). DECKERPLUS initially attempted to invoke static function pointer analysis but found the results very unacceptable (especially in a C/C++ setting), so it instead opts to solve this at runtime. The instrumentation passes the function pointer to the runtime library, which then maps

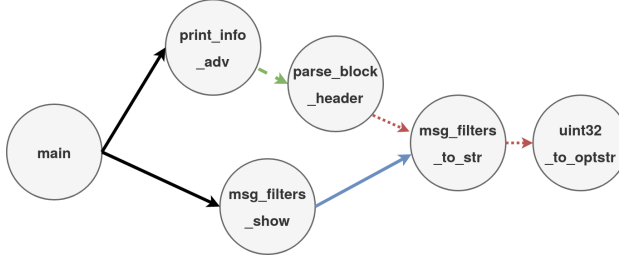


Fig. 3. Simplified callgraph from the xz data compression application. This illustrates four types of edges, each of which requires different handling by the instrumentation pass.

---

**ALGORITHM 1:** Pseudocode for DECKERPLUS's compiler pass
 

---

```

function RUN_ON_FUNC(func)
  for instr in func do
    switch instr.getType() do
      case LOOP_START
        INSERT_DECK(LOOP, instr.id)
      case DIRECT_CALL_ENCOMPASSED
        target ← instr.GET_CALL_TARGET()
        INSERT_DECK(REACHABLE, target.id)
      case DIRECT_CALL_SINGLE
        target ← instr.GET_CALL_TARGET()
        INSERT_DECK(SINGLE, target.id)
      case INDIRECT_CALL
        target_addr ← instr.GET_FUNC_PTR()
        INSERT_DECK(INDIRECT, target_addr)
    end for
  end function
  
```

---

the appropriate page(s). Indirect decks may need to be placed inside loops if static analysis fails to hoist the value outside. Such cases must be optimized to avoid drastically degrading performance (discussed at the end of Section 4.2).

Figure 3 depicts a sub-callgraph from SPEC CPU 2017's xz, a data compression application [16]. It illustrates all but the **indirect** case. Each node in the figure is a function, and each edge is a call. Only the call from `print_info_adv` to `parse_block_header` (dashed, green) is inside of a loop. The set of encompassed functions is therefore `{parse_block_header, msg_filters_to_str, uint32_to_optstr}`, and the set of non-encompassed functions is `{main, msg_filters_show, print_info_adv}`. Instrumentation will be treated as follows:

- (1) The solid-black edges from `main` require **single** decks.
- (2) The dashed-green edge from `print_info_adv` requires a **loop** deck; instrumentation will be at the loop preheader that dominates the call to `parse_block_header`.
- (3) The solid-blue edge from `msg_filters_show` requires a **reachable** deck.
- (4) The dotted-red edges from `parse_block_header` and `msg_filters_to_str` will have no instrumentation, because they are encompassed.

The basic pseudocode for the compiler instrumentation pass is shown in Algorithm 1. It shows the function `run_on_func`, which is a hook called by the pass manager on each function. The pseudocode shows the general logic for how decks are selected and inserted. Each deck needs only



one key piece of runtime information: a unique ID generated statically for each loop or function. At runtime, the library maps this parameter to a set of functions and their corresponding pages in memory. (In the case of indirect calls, the only difference is that the runtime target address is used instead of a statically known ID.) Several details not shown in the pseudocode but that should be noted include the insertion of the deck teardown calls, the insertion of an initialization call at program start, the construction of functions' static reachability, and the construction of the encompassed function set.

DECKERPLUS's compiler instrumentation supports non-trivial C and C++ behavior. It handles recursion and **strongly connected components (SCCs)** similar to loops by adding such functions to the encompassed functions. Thus, DECKERPLUS does not instrument inside SCCs (which avoids overheads due to repeated execution). Other general features that DECKERPLUS must support include the following. Beyond call instructions in LLVM IR, it must handle invoke instructions, which are a variant of call instructions used for calls that may throw exceptions with both a normal return destination and an unwind destination (the path taken when an exception is propagated up the call stack), and landing pads, which are the entry points for exception handling code that serve as the targets of those unwind edges. DECKERPLUS correctly identifies and instruments both for decks, including the decking behavior along exception control flow cases. It handles external libraries that take and then invoke a callback to the application. It handles C++ destructors invoked when an exception is thrown (via `__cxa_throw`). It handles `libc_nonshared.a`. It handles start-up C++ code before `main`. It handles signal handlers, including `atexit` and `on_exit`. Though the details for these cases are unimportant, it is important to stress that the approach is general.

**4.1.2 Function Pointers.** We follow the callgraph construction of LLVM, which safely handles function pointers. As stated in the LLVM source code, the callgraph is implemented as a “conservative superset of all of the caller-callee relationships, which is useful for transformations;” an extension/improvement in the future could be to “prove (through pointer analysis) that an indirect call site can call only a specific set of functions” [10]. As currently implemented, however, LLVM creates two “external” nodes in a module's callgraph. The first node represents all external calls that can enter the module: If a function's linkage is external/visible outside of the module, it must have an edge from this node; and, *if a function has its address taken, it must have an edge from this node*. The second node represents all external calls that can leave the module: If a function is external to the module, it must have an edge to this node; and, *if a function invokes an indirect call, it must have an edge to this node* [10]. In other words, all functions that are either indirectly callable or that call an edge connects function pointers to one of these nodes, which is sufficient (albeit conservative) for DECKERPLUS to statically identify all functions that must be mapped when any particular deck (i.e., portion of the callgraph) is added.

As we will show in the evaluation (see Section 6.1.2), unfortunately, the static function pointer analysis in LLVM is insufficient for reducing gadgets due to its low precision. DECKERPLUS does not attempt to alleviate it with flow, field, or context sensitivity techniques. Instead, DECKERPLUS uses the dynamic target of a function pointer to enable the target function's page(s). Note that the dynamic pointer target in no-attack scenarios must be valid; thus, the applications do not crash, and the technique is still sound. Also, as mentioned earlier, we do not address the orthogonal issue of function pointer integrity but rely on other approaches such as CFI [1, 20]. Other function pointer analysis works (such as for Java virtual methods [55]) could form the basis for extending LLVM or DECKERPLUS in the future. The programming language plays an integral part in the callgraph construction. Dynamic facilities like Java's reflection or JavaScript's `eval` would have to be carefully handled if applying techniques like DECKERPLUS beyond LLVM.

**4.1.3 Linking.** At the end of the compiler pass, DECKERPLUS outputs a custom linker script. Intuitively, the goal of the linker script is to separate functions into different pages so that marking a given function as RX does not “activate” unrelated functions that reside in the same page (i.e., make unrelated functions and their gadgets available for use). Thus, this step aims to factor out such functions into separate pages. An example and pseudocode may help to understand this more clearly.

**Example:** Referring again to the callgraph in Figure 3, the deck sets for this callgraph  $Dk.*$  are:

$$\begin{aligned} Dk.S1 &= \{print\_info\_adv\} \\ Dk.S2 &= \{msg\_filters\_show\} \\ Dk.L &= \{parse\_block\_header, msg\_filters\_to\_str, \\ &uint32\_to\_optstr\} \\ Dk.R &= \{msg\_filters\_to\_str, uint32\_to\_optstr\} \end{aligned}$$

Any of these functions can arbitrarily belong to the same system page at runtime. For example, without any enforcement, `print_info_adv` and `msg_filters_show` could reside in the same page. Similarly, `parse_block_header` can occupy the same system page as either of the functions in  $Dk.R$ . That is, invoking `msg_filters_to_str` from `msg_filters_show` at runtime could inadvertently activate the gadgets in `parse_block_header`. The solution is to rely on the fact that DECKERPLUS instrumentation ensures that each deck will be mapped RX independently at runtime, and to leverage the custom linker script to avoid this security penalty. The custom linker script should separate the intersection  $Dk.L \cap Dk.R = \{msg\_filters\_to\_str, uint32\_to\_optstr\}$  into its own disjoint set. Thus, the full disjoint sets  $Dj.*$  are as follows:

$$\begin{aligned} Dj.S1 &= \{print\_info\_adv\} \\ Dj.S2 &= \{msg\_filters\_show\} \\ Dj.L.1 &= \{parse\_block\_header\} \\ Dj.I.LR &= \{msg\_filters\_to\_str, uint32\_to\_optstr\} \end{aligned}$$

where  $Dj.I.LR$  is the disjoint set formed by  $Dk.L \cap Dk.R$  and  $Dj.L.1$  is the disjoint set formed by  $Dk.L \setminus (Dk.L \cap Dk.R)$ . Each of these disjoint sets will be page-aligned by the custom linker script. Such a separation allows finer control over the activation of decks, although it increases the code size.

**Pseudocode:** Pseudocode for creating these disjoint sets is shown in Algorithm 2. The algorithm begins with the deck sets ( $Dk.*$  in our example). A deck set corresponds directly to 1 of the 4 types of decks: the function of a **single** deck forms a singleton; the functions of a **loop** deck form their own set; any encompassed function that can be called from some non-loop path has itself and any **reachable** functions as part of a set; and any functions that have their addresses taken and can be invoked by some **indirect** call form a set with their statically reachable callees. The algorithm begins with a list of these sets and then iterates, attempting to separate functions into their own disjoint sets ( $Dj.*$  in our example). To find the disjoint sets, each pair of the decks is intersected with each other. If the intersection is non-null, those shared members are removed from the pair of decks and form their own disjoint set. This pairwise intersection-removal process is repeated until no more disjoint sets can be formed. Once the disjoint sets are known, each one is assigned its own page-aligned section in the linker script. Though this implementation is relatively clear, it is too slow for practical use. In the actual DECKERPLUS implementation, it is optimized with a bottom-up approach: Whenever a new deck set is created within the pass, it is immediately pushed into the working disjoint sets (and any intersections are applied).

**4.1.4 Precision and Recall.** In the context of DECKERPLUS, we define false positives (precision) as alarms for never-occurring attacks. This case is not possible under DECKERPLUS; DECKERPLUS

**ALGORITHM 2:** Pseudocode for creating disjoint sets for DECKERPLUS's custom linker script

---

```

function CREATE_DISJOINT_SETS(deck_sets)
  disjoint_sets  $\leftarrow \emptyset$ 
  while !deck_sets.EMPTY() do
     $A \leftarrow \text{deck\_sets}[0]$ 
     $\text{tmp} \leftarrow \text{deck\_sets}[1 : ]$ 
    deck_sets  $\leftarrow \emptyset$ 
    for  $B$  in tmp do
       $I \leftarrow A \cap B; A_I \leftarrow A \setminus I; B_I \leftarrow B \setminus I$ 
      if  $|I| == 0$  then
        deck_sets.PUSH( $B$ )
      else
        if  $|B_I| > 0$  then
          deck_sets.PUSH( $B_I$ )
        end if
        deck_sets.PUSH( $I$ )
         $A \leftarrow A_I$ 
      end if
    end for
    disjoint_sets.PUSH( $A$ )
  end while
  return disjoint_sets
end function

```

---

faithfully follows program semantics under caller-callee relations, so any control flow outside a deck under a no-attack scenario is impossible. We define false negatives (recall) as attacks that occur but are missed. False negatives are possible under DECKERPLUS (see Section 6.1.4 for experimental results). One way to quantify it is as the additional code surface mapped RX by DECKERPLUS but never executed. This can happen because the mapped code is a (tight) overapproximation of the forthcoming code at any particular program point. The false negative rate is also related to the soundness properties and function pointer analysis of DECKERPLUS. In terms of soundness, an underapproximated mapping of the forthcoming code would reduce false negatives, but it would be unsound. In terms of function pointer analysis, replacing DECKERPLUS's dynamic technique with pure static analysis would lead to much larger code mappings and, therefore a higher false negative rate.

## 4.2 Runtime Component

The runtime support is exposed as a library to the application. It enables and disables pages by marking them RX or RO. The API is shown in Listing 2. Although not shown in the listing, these API calls have a corresponding deck teardown call to remap the relevant pages RO (and initialization has a corresponding destroy call). The API calls align directly with the four types of decks mentioned previously (single, loop, reachable, and indirect), plus library initialization. They also have a corresponding deck teardown call to remap the relevant pages RO.

---

```

int deck_init(void);
int deck_single(int callee_func_id);
int deck_loop(int loop_id);
int deck_reachable(int callee_func_id);
int deck_indirect(long long callee_addr);

```

---

Listing 2. DECKERPLUS's runtime API.

Two essential steps are necessary for library initialization. The first is identifying the binary's base address. Function offsets are known at build-time, but at runtime DECKERPLUS still needs to determine the text section's base address. The second step is to protect all text pages by marking them RO. This happens at the start of main, and main is left RX.

Regarding the primary API endpoints, `deck_single` takes the function ID of an impending callee as an argument. The runtime uses the ID to look up the actual page addresses of this function, and it marks them as RX. When that function returns, `deck_single_end` will mark the pages as RO. `deck_reachable` is similar, taking a callee function ID as its only argument. Because the callee is an encompassed function, though, all pages of statically reachable functions must be marked RX, as well. These are compile-time known, so the runtime library only needs to issue a map lookup to find which set of functions to mark RX for that particular callee. `deck_loop` takes a single loop ID parameter as its argument. The compiler assigns a unique loop ID to each interprocedural, outermost loop in the program. It is a simple lookup to find which functions are part of that loop and map them RX. `deck_indirect` takes a runtime address as its argument. The library maps this runtime address to the corresponding function and determine whether that function is an encompassed or non-encompassed function. If it is encompassed, then the library leverages its own `deck_reachable` support for that function. Similarly, non-encompassed functions are handled by the `deck_single` support.

DECKERPLUS maintains a reference counter for the text pages. When a function is needed, the reference count for each of its pages is incremented; when that function is no longer needed, the reference count for each of its pages is decremented. Whenever a page's count changes from 0 to 1, the page must be marked RX; and whenever a page's count changes from 1 to 0, it can be marked RO again. We define the set of pages at runtime with reference counts greater than 0 as the **available pages**. Adding a deck at runtime will either increase the cardinality of the available pages (if new pages are needed), or does not affect its size (if all required pages are already available); the opposite holds for removing a deck.

There is one critically important optimization we make that is called **indirect deck caching (IDC)**. When there is an indirect call inside a loop, IDC inlines a check against its address, and that deck is "cached" for the duration of the loop. We find that this reduces slowdown by up to 80% in some cases. It is also an argument against the static treatment of function pointers, which is fast but enables too many gadgets at loop headers (~40% worse).

## 5 Compiler Extension

On top of the baseline DECKERPLUS, we implemented two compiler extensions: Function Cloning and Inlining. These techniques are separately enabled and work before the linking stage.

### 5.1 Function Cloning

As previously stated, decks are heavily organized around loops due to the significant performance degradation they can cause. If this performance degradation were not a concern, DECKERPLUS could create decks containing a single function for the best security results. DECKERPLUS handles loops via loop decks where all interprocedural reachable functions within the loop are placed inside a single loop deck. Recall from Section 4.1.1 that an encompassed function is reachable through the callgraph by some function that is called within a loop through a caller-callee relation. When an encompassed function is called, DECKERPLUS will map the entire loop deck. To see why this is not always ideal for security, we present a simple example in Figure 4.

Figure 4 shows a sub-callgraph from SPEC CPU 2017's `povray`, a ray tracing application [16]. The dashed black arrow represents the call path originating from the loop header, `Cooperatev`, while the red arrow represents a call from outside the loop into `Send_InitInfo`. In the baseline DECKERPLUS,

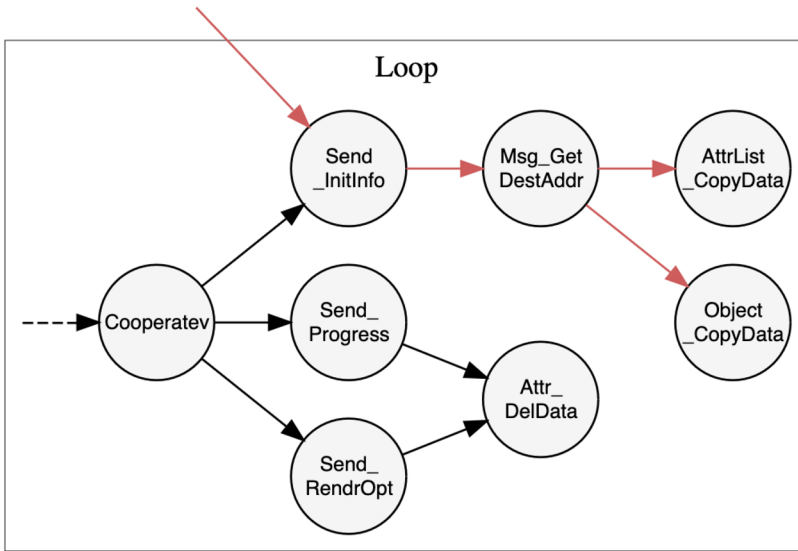


Fig. 4. Simplified callgraph from the povray ray tracing application. This illustrates a loop deck that contains the encompassed function `Send_InitInfo`.

all functions reachable from the loop are assigned to the same loop deck. Consequently, because `Send_InitInfo` belongs to this loop deck, an external call to `Send_InitInfo` also activates the entire loop deck, even though execution did not originate from the loop. Ideally, when execution enters from the external caller, only the functions along that call path (e.g., from `Send_InitInfo` to `AttrList_CopyData` and `Object_CopyData`) would be grouped into smaller decks. However, using these fine-grained decks for loop iterations would require repeatedly enabling and disabling them, incurring significant runtime overhead. Function cloning separates the loop and non-loop calling contexts, allowing the external path to use fine-grained decks while preserving the efficient loop deck for iterative execution, thereby reducing the attack surface without introducing excessive overhead.

Function cloning allows us to create decks that contain a subset of the original loop deck without affecting the contents of the loop deck. Returning to our example in Figure 4, we would clone the subset of functions connected via red arrows to create a new subgroup of functions shown in Figure 5. These function clones are then decked with single decks. The original loop deck will be mapped when the loop is entered via the loop head. Function cloning allows more granularity in the function mapping when dealing with encompassed functions.

**Pseudocode:** The pseudocode for cloning the functions is shown in Algorithm 3 implemented as part of the LLVM compiler component. DECKERPLUS finds all encompassed functions and checks if they can be called from a non-encompassed function. If so, it means a function not inside a loop calls a function inside one. Then, we add all the reachable functions from the encompassed function. We will clone them all and replace the called functions at the callsites with the clone functions. From our example in Figure 4, the encompassed function is `Sed_InitInfo`, and the non-encompassed function is the external function. The reachable functions are `Msg_Get_DestAddr`, `Object_CopyData`, and `AttrList_CopyData`. We clone `Sed_InitInfo` and create `Sed_InitInfo_copy` and replace the called function in the external function to `Sed_InitInfo_copy`. Recursively, we will replace the called functions in the clones with the called functions clones so `Sed_InitInfo_copy` will call `Msg_Get_DestAddr_copy` and so on for the other ones.

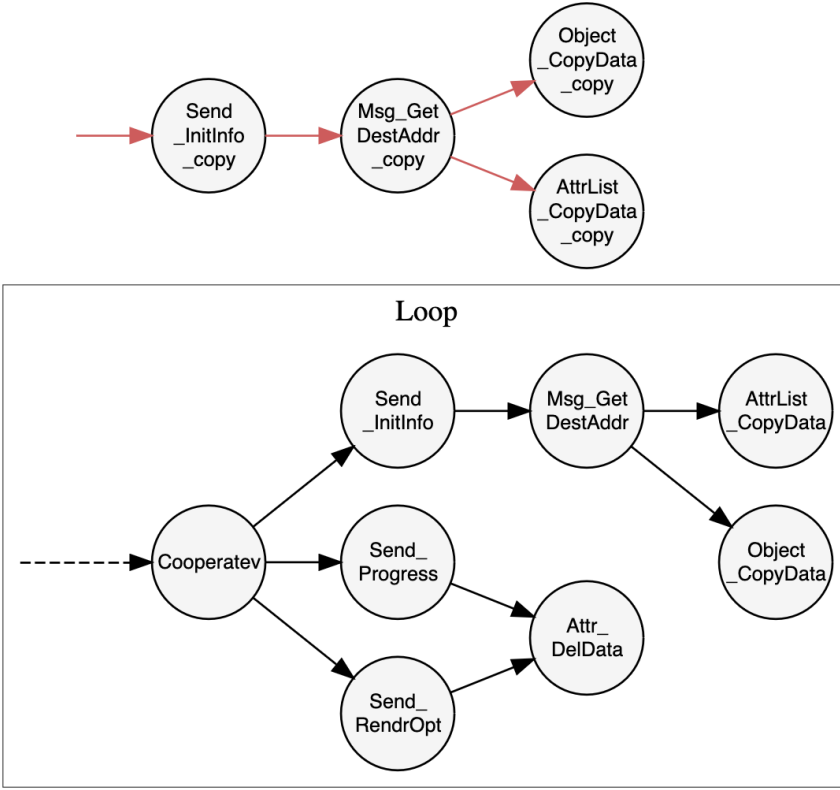


Fig. 5. Cloned sub-callgraph from povray. This illustrates a reachable deck that contains the clones of an encompassed function and all interprocedural reachable functions.

## 5.2 Inlining

Though function cloning helps with specific cases, it can increase binary size by duplicating selected functions. Inlining provides a complementary transformation. At a callsite, the caller is the function making the call and the callee is the function being called. Inlining replaces a selected callsite with a copy of the callee body, so the callee code executes as part of the caller rather than through a separate function call. We apply LLVM’s optimizer with different inlining thresholds before running DECKERPLUS’s instrumentation pass. LLVM’s InlinerAdvisor estimates the cost of the callee after inlining and subsequent optimization opportunities [57]. If the estimated cost is below the configured threshold, the callsite may be inlined. The threshold can be set with `-inline-threshold=`; LLVM’s default threshold is 225. In our scheme, we evaluate thresholds from 100 to 1000 and report the threshold that gives the strongest gadget reduction for each benchmark.

Inlining affects DECKERPLUS primarily by changing which code pages become available at runtime. We use available pages to mean the pages present in a runtime available-page snapshot, i.e., the pages marked executable by the currently active decks. A page may still exist in the binary but never appear in any available-page snapshot, meaning it is not executable during the observed execution.

This mechanism is important because inlining can remove the runtime callsite that would otherwise cause a callee’s deck to be activated. If a function is called normally, DECKERPLUS inserts instrumentation around the call and the callee’s pages may be added to the available set for the duration of that call. If the function is inlined, no separate call to that callee remains at runtime. The



**ALGORITHM 3:** Pseudocode for DECKERPLUS's function cloning

---

```

function CLONE_FUNCS(unencompassed_funcs, encompassed_funcs)
  funcs_to_clone  $\leftarrow \emptyset$ 
  clone_funcs  $\leftarrow \emptyset$ 
  clone_map  $\leftarrow$ 
  for callsite in getAllCallsites() do
    caller  $\leftarrow$  callsite.getCaller()
    callee  $\leftarrow$  callsite.getCalledFunction()
    if callee  $\in$  encompassed_funcs && caller  $\in$  unencompassed_funcs then
      funcs_to_clone.push(callee)
      funcs_to_clone.union(callee.getReachable())
    end if
  end for
  for func in funcs_to_clone do
    clone_func  $\leftarrow$  clone(func)
    clone_map[func] = clone_func
    clone_funcs.push(clone_func)
  end for
  for callsite in getAllCallsites() do      ▷ After cloning, getAllCallsites will have the clone function's
  callsites too
    caller  $\leftarrow$  callsite.getCaller()
    callee  $\leftarrow$  callsite.getCalledFunction()
    if callee  $\in$  encompassed_funcs && caller  $\in$  unencompassed_funcs then
      callsite.replaceCalledFunction(clone_map[callee])
    else if callee  $\in$  funcs_to_clone && caller  $\in$  clone_funcs then
      callsite.replaceCalledFunction(clone_map[callee])
    end if
  end for
end function

```

---

copied code executes within the caller's context, and DECKERPLUS does not enter a separate deck activation for the original callee body. As a result, the original pages containing that callee may never be made executable even though the function's code is still physically present in the binary.

This explains why function removal is not the main source of improvement. Dead-code elimination can remove some functions after inlining, but only when every callsite has been inlined away and the function cannot be reached through a function pointer. This is a restrictive condition. Many functions remain in the binary because at least one callsite remains, or because the function can be called through a pointer at runtime, so the compiler cannot prove it is safe to remove. Redis illustrates this point. In the most aggressive inlining configuration, only 73 of 6,360 functions are removed, roughly 1.1%. However, the average available pages per snapshot drops substantially for the best inlined variants, showing that the improvement comes mainly from runtime page availability rather than wholesale function removal.

Instead, the main effect is that inlining substantially reduces the number of pages that are executable in each runtime context. This can occur even when the final binary becomes larger. Inlining often duplicates code because callee bodies are copied into their callers, while the original callee body may still remain in the binary. Therefore, the total number of gadgets in the binary can increase. However, gadgets on pages that never appear in an available-page snapshot are not available to an attacker under DECKERPLUS's enforcement because those pages are not executable during the observed execution. The relevant metric is therefore not the total number of gadgets present in the binary, but the number of gadgets on available pages in each runtime snapshot.

The `lighttpd` page-map data provides a clear example of this effect. In the baseline `DECKERPLUS` variant, the request-serving path reaches an available-page snapshot containing 79 pages. With inlining enabled, the corresponding request-serving snapshot contains only 2 pages. The total binary gadget count does not explain this improvement: the inlined `lighttpd` variants contain more total gadgets than the baseline, increasing from 2,218 gadgets in the baseline to 2,580–2,923 gadgets across the inlined variants. Thus, inlining improves gadget reduction not by removing gadgets from the binary, but by preventing many gadget-containing pages from becoming executable during the measured execution.

This also explains why higher inlining thresholds are not always better. The threshold controls inlining for all direct calls seen by the optimizer, not only the indirect calls exposed by `DECKERPLUS`'s preprocessing. At low and moderate thresholds, inlining can remove many runtime deck activations and keep callee pages out of the available-page snapshots. At higher thresholds, larger functions may be copied into callers, which can pull additional code into the executed context and cause more pages to become available. Thus, aggressive inlining can sometimes partially reverse the benefit by increasing the number of available pages per snapshot.

We therefore treat inlining as a transformation that improves gadget reduction mainly by changing runtime page availability, rather than as a direct return-instruction removal defense. We also do not use the number of compiler-generated return instructions as a proxy for ROP gadget availability. On x86, useful ROP gadgets can arise from unintended instruction boundaries, so removing source-level call and return boundaries does not by itself imply fewer usable gadgets. As described in Section 6, our evaluation measures the gadgets available on executable pages in each runtime snapshot. Under this measurement, inlining improves `DECKERPLUS` because it reduces the set of executable pages in each runtime context, while dead-code elimination provides only a secondary and much smaller contribution.

## 6 Evaluation

We evaluate `DECKERPLUS` on two commodity machines. Linux experiments run in an Ubuntu 18.04 LTS Docker container with LLVM v11.0.0 on an Intel Xeon Gold 5117 CPU. The Windows experiments in Section 6.6.2 use an AMD Ryzen 7 2700X with 32GB RAM, Windows 10 Education v21H2, and LLVM v11.0.0 built with `cl.exe v19.31.31104`.

Our workloads include SPEC CPU 2017 [16], 10 GNU coreutils programs [15], `nginx v1.20.1` [37], `Redis v8.6.3` [44], `lighttpd v1.4.82` [30], and 6 `xpdf v4.03` command-line tools [63]. Many have been used in prior debloating and attack-surface-reduction evaluations, enabling direct comparison, while `Redis`, `lighttpd`, and `xpdf` broaden the study with additional server and document-processing applications. Unless otherwise stated, `DECKERPLUS` uses the IDC optimization described in Section 4.2; baseline binaries use the same compiler and optimization settings without `DECKERPLUS`.

Our evaluation addresses the following questions:

- (1) What runtime slowdown and code-size growth does baseline `DECKERPLUS` introduce?
- (2) How much does baseline `DECKERPLUS` reduce dynamically available gadgets?
- (3) How much do function cloning and inlining improve gadget reduction?
- (4) Can `DECKERPLUS` break practical gadget chains and restrict JOP gadgets on Linux and Windows?

**Security:** With `DECKERPLUS`, the gadgets that are available to an attacker change dynamically during runtime based on which pages `DECKERPLUS` has mapped RX, i.e., based on the **available pages** (discussed in Section 4.2).

To calculate these values, we start with the total number of gadgets in the application. This is the sum of ROP, JOP, COP, and special-purpose gadgets (reported by `GadgetSetAnalyzer` [8]):

$$T = R + J + C + S \quad (1)$$

The reduction of total gadgets for some set of available pages  $AP$  during runtime is given by the equation:

$$reduction_{AP} = \frac{T - T_{AP}}{T} * 100 \quad (2)$$

—where  $T$  is the total number of gadgets in the baseline application, and  $T_{AP}$  is the total number of gadgets in that available page set. The average reduction over all available page sets is given by the formula:

$$avg\_reduction = \frac{\sum reduction_{AP}}{num\_APs}$$

—where  $reduction_{AP}$  is the total gadget reduction for some available page set  $AP$  versus the baseline. We sum over all such reductions and divide by the number of available page sets  $num\_APs$ .

We enable DECKERPLUS's runtime logging and execute each program over all test inputs. Every DECKERPLUS API call records the current set of available pages. We then identify the gadgets contained in each recorded page set. Identical available-page sets are treated as the same execution state for gadget counting.

The set of available pages with the least proportion of reduced gadgets is marked as the “minimum reduction,” and vice versa for maximum reduction. The average reduction is the average proportion of gadgets reduced by each set of available pages over the logs.

Later in this evaluation, we talk about breaking gadget chains and the analysis was done on both Linux and Windows. For this case, we only ran baseline DECKERPLUS on Windows but we looked at the baseline DECKERPLUS and extensions on Linux.

## 6.1 SPEC CPU 2017

SPEC CPU 2017 [16] is a standard suite for CPU-bound performance evaluation and includes diverse C and C++ applications from domains such as route planning, discrete-event simulation, video compression, alpha-beta tree search, molecular dynamics, and ray tracing. This makes it useful for measuring both DECKERPLUS's performance overhead and its security impact.

**6.1.1 Performance.** Figure 6 reports normalized performance. Each benchmark is compiled at -O3 and evaluated both without and with DECKERPLUS. Baseline DECKERPLUS has a worst-case slowdown of 17.7% on omnetpp. Cloning and inlining have worst cases of 22.8% and 21.7%, respectively. Some benchmarks show slight speedups, including blender under baseline DECKERPLUS and namd with inlining. Such effects are common in instrumentation-based systems because changes in function placement and memory alignment can improve performance [26, 40]. Average slowdowns are 6.46%, 6.39%, and 6.48% for baseline, cloning, and inlining, showing that the extensions add negligible overhead over baseline DECKERPLUS.

Performance generally improves as the inlining threshold increases until an inflection point, after which gains level off or reverse, as shown for gcc and imagick in Figure 7. Inlining enables additional -O3 optimization and reduces the number of DECKERPLUS runtime calls. For imagick, performance improves by 19% relative to baseline after the threshold increases sufficiently. Because our primary goal is gadget reduction, however, we select the threshold that maximizes that metric. At the selected thresholds, performance is similar to or better than baseline DECKERPLUS except for blender and povray, which incur 13% and 21% overhead, respectively. For xalancbmk, LLVM's inliner fails above a threshold of 500, so we use 500.

For comparison, BlankIt reports 18% overhead on SPEC CPU 2006 while debloating libraries. Razor reports 1.7% average and 16% worst-case overhead on SPEC CPU 2006. Piece-wise adds negligible load-time overhead but also targets only libraries. Given that DECKERPLUS operates on whole applications, its 6.39–6.48% average slowdown on SPEC CPU 2017 is reasonable relative to these approaches.

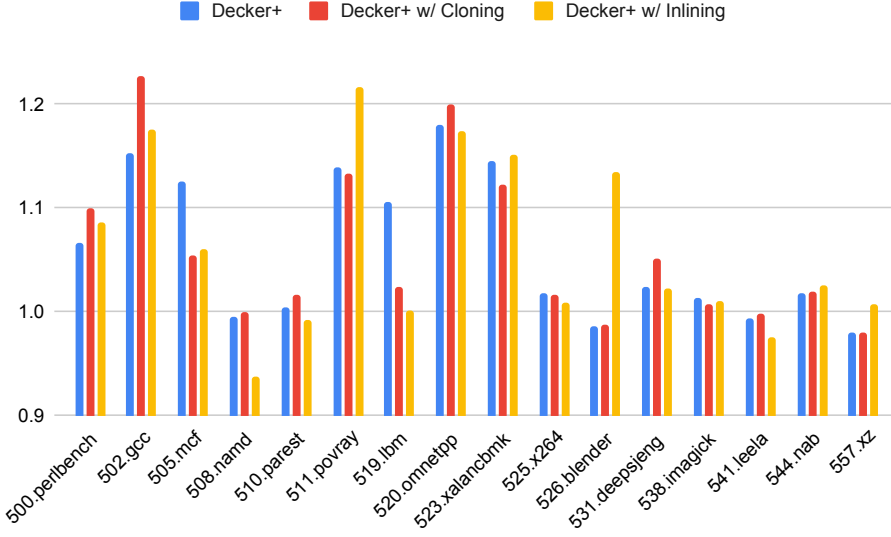


Fig. 6. Slowdown for SPEC CPU 2017 using DECKERPLUS and its extensions.



Fig. 7. Performance improvement for SPEC CPU 2017 with DECKERPLUS as the inlining threshold increases.

**6.1.2 Security.** Table 2 reports per-benchmark raw gadget counts and gadget reductions. SPEC binaries range from 145 gadgets in *lbm* to 105,201 in *blender*. Across these benchmarks, baseline DECKERPLUS reduces total gadgets by 70.3% on average, with the custom linker script contributing approximately 15% of the reduction. Total gadget reduction is representative of the individual ROP, JOP, COP, and special-purpose categories. For example, ROP gadgets are reduced by 74.4% on average.

To evaluate the importance of dynamic function-pointer resolution, we also measure a DECKERPLUS variant that relies on LLVM’s built-in static function-pointer analysis. Its average total gadget reduction falls to 39.9%, with average minimum and maximum reductions of 27.6% and

Table 2. Total Gadget Reduction Across All Benchmarks (Higher is Better)

| Suite        | Benchmarks      | Raw Gadgets | Decker+ |      |      | Decker+ w/ Cloning |       |      | Decker+ w/ Inlining |      |       |      |
|--------------|-----------------|-------------|---------|------|------|--------------------|-------|------|---------------------|------|-------|------|
|              |                 |             | Min     | Max  | Avg  | Min                | Max   | Avg  | Inline Thr.         | Min  | Max   | Avg  |
| SPEC         | perlbench       | 7,452       | 47.9    | 98.2 | 61.2 | 50.4               | 98.9  | 64.2 | 800                 | 58.3 | 99.7  | 68.7 |
|              | gcc             | 41,654      | 42.6    | 99.5 | 59.7 | 47                 | 99.7  | 63.8 | 900                 | 40.2 | 99.8  | 56.8 |
|              | mcf             | 285         | 61.7    | 63.6 | 62.8 | 60.8               | 64.1  | 62.7 | 900                 | 36   | 65.3  | 57.2 |
|              | namd            | 1,826       | 76.3    | 94.7 | 89.8 | 75.3               | 94.3  | 87.3 | 500                 | 84.6 | 97.9  | 94.4 |
|              | parest          | 26,512      | 77.1    | 99.7 | 93.5 | 79.9               | 99.7  | 93.1 | 900                 | 78   | 99.7  | 92.6 |
|              | povray          | 5,822       | 36.7    | 98.8 | 47   | 64.1               | 97.9  | 79.9 | 900                 | 67.3 | 84.2  | 76.9 |
|              | lbm             | 145         | 35.4    | 47.2 | 42.2 | 45.2               | 56.8  | 53.4 | 700                 | 51   | 99.6  | 74   |
|              | omnetpp         | 18,848      | 51.7    | 98.7 | 79   | 52                 | 99.2  | 74.5 | 1000                | 51   | 99.6  | 74   |
|              | xalancbmk       | 40,532      | 58.9    | 99.7 | 71.5 | 59.6               | 99.7  | 80   | 500                 | 69.1 | 99.9  | 80.6 |
|              | x264            | 3,920       | 19.7    | 96.6 | 33.4 | 19.9               | 96.7  | 65.8 | 900                 | 25.5 | 98.9  | 38.3 |
|              | blender         | 105,201     | 73.7    | 99.8 | 96.9 | 71.8               | 99.8  | 98.1 | 400                 | 81   | 100.0 | 98.1 |
|              | deepsjeng       | 484         | 59.9    | 71.9 | 68.3 | 62.3               | 73.1  | 70   | 700                 | 77.5 | 89.3  | 87   |
|              | imagick         | 8,412       | 35.6    | 99.2 | 68.5 | 32.6               | 99.2  | 68   | 900                 | 34.5 | 99.4  | 63.6 |
|              | leela           | 813         | 64.1    | 88.5 | 83.4 | 81.5               | 86.3  | 84.7 | 900                 | 97   | 97.8  | 97.5 |
|              | nab             | 855         | 69.8    | 92.1 | 88.2 | 73                 | 93.7  | 87.4 | 900                 | 81.8 | 96.2  | 93.5 |
|              | xz              | 1,713       | 60.5    | 94.4 | 79.5 | 62.8               | 94.2  | 78.5 | 1000                | 72.9 | 99.5  | 87.6 |
|              | Average         | –           | 54.5    | 90.2 | 70.3 | 58.4               | 90.6  | 75.5 | –                   | 62.9 | 96.1  | 77.6 |
| CoreUtils    | bzip2           | 339         | 73.6    | 77.9 | 76.8 | 73.4               | 85.5  | 81.6 | 900                 | 90.4 | 91.3  | 91   |
|              | gzip            | 360         | 46.3    | 74.3 | 68.4 | 45.5               | 75.6  | 69.3 | 1000                | 52.9 | 93.7  | 86   |
|              | tar             | 1,335       | 56      | 96   | 93.4 | 58.8               | 94.1  | 91.5 | 100                 | 88.9 | 97.7  | 97   |
|              | grep            | 1,087       | 60.3    | 87.8 | 79.2 | 68.3               | 94.7  | 86.4 | 800                 | 71.7 | 97.7  | 88.4 |
|              | chown           | 1,809       | 95.6    | 97.3 | 96.8 | 92.6               | 94.8  | 94.1 | 100                 | 96.8 | 98.9  | 98.3 |
|              | date            | 2,133       | 93.8    | 96.6 | 96   | 95.8               | 98    | 97.3 | 900                 | 98.3 | 99.5  | 99.2 |
|              | mkdir           | 757         | 86.7    | 93   | 91.3 | 92.1               | 94.6  | 93.8 | 500                 | 94.6 | 96.8  | 96   |
|              | rm              | 1,827       | 85      | 95.6 | 94.2 | 87.2               | 95.7  | 94.6 | 1000                | 88.9 | 99.4  | 97.3 |
|              | sort            | 1,871       | 78.8    | 93.6 | 91.7 | 81                 | 94.4  | 92.7 | 500                 | 88.2 | 97.1  | 95.3 |
|              | uniq            | 1,751       | 94.5    | 97.8 | 97.1 | 92.6               | 95    | 94.5 | 900                 | 96.9 | 98.8  | 98   |
|              | Average         | –           | 77.1    | 91.0 | 88.5 | 78.7               | 92.2  | 89.6 | –                   | 86.8 | 97.1  | 94.7 |
| Applications | nginx           | 3,844       | 51.1    | 96.3 | 81.3 | 51.1               | 96.3  | 81.3 | 800                 | 76.6 | 99.4  | 92.3 |
|              | redis-server    | 21,505      | 87.7    | 99.5 | 96.3 | 97.7               | 99.7  | 99.3 | 700                 | 95.7 | 99.9  | 99.6 |
|              | lighttpd        | 2,138       | 56.3    | 97.5 | 79.8 | 96.7               | 100.0 | 97.5 | 300                 | 99.3 | 99.3  | 99.3 |
|              | xpdf-pdf-totext | 8,098       | 98.3    | 98.8 | 98.6 | 98.1               | 99.8  | 99.4 | 200                 | 99.6 | 100.0 | 99.9 |
|              | xpdf-pdf-info   | 7,761       | 98.8    | 98.9 | 98.9 | 90.0               | 98.8  | 97.7 | 700                 | 99.7 | 99.7  | 99.7 |
|              | xpdf-pdf-fonts  | 7,738       | 99.3    | 99.3 | 99.3 | 97.5               | 99.1  | 98.8 | 900                 | 99.9 | 99.9  | 99.9 |
|              | xpdf-pdf-detach | 7,736       | 98.5    | 98.8 | 98.7 | 93.5               | 99.8  | 98.9 | 1000                | 99.6 | 99.8  | 99.8 |
|              | xpdf-pdf-images | 7,767       | 98.5    | 98.8 | 98.7 | 91.2               | 99.8  | 98.3 | 100                 | 99.8 | 100.0 | 99.9 |
|              | xpdf-pdf-tops   | 9,922       | 92.9    | 98.9 | 96.9 | 94.8               | 100.0 | 99.4 | 700                 | 99.5 | 99.9  | 99.8 |

Raw Gadgets is the total number of gadgets in the unmodified binary. Inline Thr. reports the inlining threshold with the best average reduction.

41.5%, respectively. This is nearly half as effective as the dynamic approach, demonstrating that runtime target resolution is crucial for gadget reduction. Function cloning increases the average reduction to 75.5%, while inlining raises it further to 77.6%. The average minimum and maximum reductions improve by 8.1 and 4.4 percentage points, respectively.

Direct comparison with prior work is difficult because the evaluated code and reported metrics differ. Piece-wise reports 72.88% average total gadget reduction for musl-libc on SPEC CPU 2006 but does not reduce whole applications. BlankIt reports 97.8% average ROP gadget reduction for glibc and other libraries on SPEC CPU 2006, but likewise does not target application code. Razor reports 68.19% application code reduction rather than gadget reduction. Thus, DECKERPLUS provides stronger application-level reduction than Razor without sacrificing soundness. Relative to

library-only systems, its gadget reduction is comparable to the load-time Piece-wise approach but lower than BlankIt's on-demand runtime reduction.

**6.1.3 Compilation Time.** We measure the time required by DECKERPLUS's LLVM pass and the complete build process. Without cloning, the pass completes in under one second for 11 of 16 SPEC benchmarks, under ten seconds for two, and in 17, 36, and 186 seconds for the remaining three. The 186-second outlier is gcc. Cloning increases pass time because it creates additional functions. Most benchmarks that previously completed in under one second remain under ten seconds, the next two require 20–30 seconds, two require 50–60 seconds, and gcc requires 4–5 minutes because it produces 3,681 clones (Table 3). Inlining does not affect pass time because its threshold is configured during -O3 optimization before the DECKERPLUS pass. Overall, DECKERPLUS increases complete binary build time by 9.3% relative to baseline.

**6.1.4 False Negative Rate.** We estimate false negatives by tracing each deck addition and measuring how much of the newly enabled code surface is subsequently executed. On average, 88.0% of enabled pages and 86.3% of enabled functions are used. Thus, DECKERPLUS enables approximately 12% additional page surface that remains unused and could potentially be available to an attacker. This is only an approximation because unused executable code does not necessarily contain a useful gadget-chain component. Sections 6.5 and 6.6 therefore evaluate whether DECKERPLUS removes complete gadget chains.

## 6.2 GNU Coreutils

We measure our technique on a subset of GNU coreutils. This package contains roughly 100 tools, including grep, mkdir, and rm. These utilities are relevant to software debloating for several reasons, including their real-world ubiquity and their history of CVEs. Chisel and Razor also report results for coreutils that DECKERPLUS can compare against.

The authors of Razor made their tool available [17], so we use the same application versions and inputs as them. Their inputs were designed to cover the same functionality tested by Chisel. We use only the test inputs, as we do not require any training. The number of inputs per benchmark ranges between 17–40, and the number of options that any given input may exercise ranges from 1–7 (see [41] for more details). Runtime overheads are negligible for coreutils. Every test completes in under 1 second and is trivially performant. (In contrast, SPEC CPU 2017 tests each take 3–10 minutes).

As with SPEC, Table 2 reports both the raw gadget counts and total gadget reduction numbers. The coreutils binaries are smaller than most SPEC applications, with raw gadget counts ranging from 339 for bzip2 to 2,133 for date.

The worst-case scenario occurs at one point during gzip, where 46.3% of the application's gadgets are unavailable. The best case occurs for uniq at 97.8%. With cloning and inlining, it goes up 89.6% and 92.2%. The worst-case is still gzip with 45.5% and 54.2% with the best being date with 98% and 99%. Razor and Chisel achieve 61.9% and 85.1% ROP gadget reduction on coreutils, respectively. Thus, DECKERPLUS fares better than two other state-of-the-art techniques that reduce ROP gadgets in application code, and more importantly, does not compromise soundness.

## 6.3 Applications

In addition to SPEC CPU 2017 and GNU coreutils, we evaluate DECKERPLUS on standalone application benchmarks. These include three server workloads, nginx, Redis, and lighttpd, as well as six command-line tools from xpdf. These applications broaden the evaluation beyond benchmark suites and small command-line utilities while still exercising realistic workloads.

*nginx.* nginx is by some metrics the most popular web server today [60]. It is used to serve web content, as a reverse proxy, and as a load balancer. As a common multitool in today's web



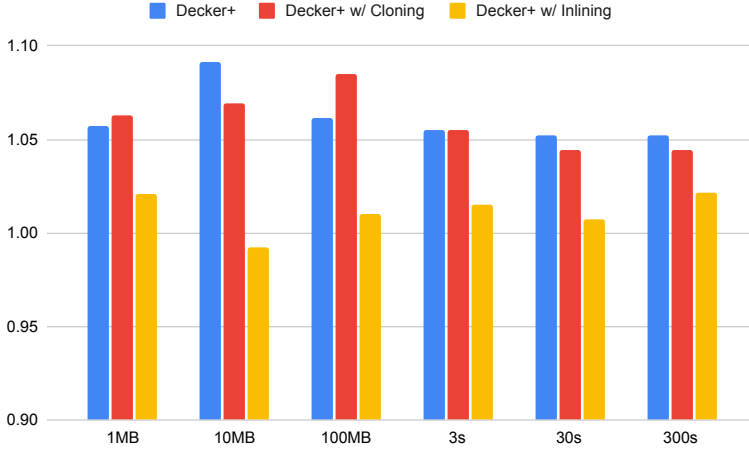


Fig. 8. Transfer/sec degradation for nginx using DECKERPLUS.

infrastructure, security is a real concern for nginx. It is multithreaded and multiprocessed, unlike the SPEC benchmarks and coreutils applications we have evaluated, and this can break or stress frameworks. nginx's performance is a critical factor in specific deployments, so it is essential that any security techniques not interfere too heavily with it. It has also recently been evaluated by another debloating technique, BlankIt, which will serve as a good comparison point. For all these reasons, we choose to evaluate nginx with DECKERPLUS.

We faithfully reproduce the security and performance experiments described in the BlankIt evaluation [40]. We use the same nginx workload generator, wrk, which runs 12 threads in parallel and creates 400 concurrent connections to the server. For the performance experiment, there are four inputs: the home page of Wikipedia, and three randomized binaries of 1 MB, 10 MB, and 100 MB. The performance experiment includes two separate tests. In the first test, wrk requests the Wikipedia home page for 3s, then 30s, then 300s. The experiment tests that nginx can serve a normal-sized page (80 KB) under high load for extended periods of time without degrading. In the second test, wrk requests the 1MB binary for 30s, then the 10 MB binary for 30s, and finally the 100 MB binary for 30s. This experiment tests that there is minimal degradation as the request size scales. For the security experiment, only the home page of Wikipedia is used. wrk makes requests for 30s.

Figure 8 shows the normalized performance result. The slowdown is reported as the transfer/sec degradation, normalized against the baseline.

DECKERPLUS achieves 1.062x slowdown on average. These results are almost identical to the original DECKERPLUS's results. This is because no functions are cloned in nginx, so the binary is exactly the same as baseline DECKERPLUS. We believe this is due to the large number of indirect function calls nginx utilizes. Function cloning is only performed on direct function calls that LLVM can reason about for safety purposes. Thus, nginx does not gain any security benefit from function cloning and has performance degradation. DECKERPLUS with function cloning is not effective on programs with heavy use of indirect pointers. On the other hand, the other extension, inlining, improves on performance having an average slowdown of 1.011x. This is primarily due to inlining combining functions and enabling further optimizations, thereby reducing the number of DECKERPLUS calls.

The total gadget reduction on nginx is 81.3% on average (51.1% and 96.3% minimum and maximum, respectively). With inlining, the gadget reduction improves to 88.6% (65.9% and 98.3% minimum

Table 3. Total Number of Functions Cloned for Each Benchmark, Grouped by Benchmark Suite or Standalone Applications

| SPEC      | Clones | Coreutils | Clones | Applications   | Clones |
|-----------|--------|-----------|--------|----------------|--------|
| perlbench | 631    | bzip2     | 30     | nginx          | 0      |
| gcc       | 3681   | chown     | 17     | redis-server   | 848    |
| mcf       | 2      | date      | 108    | lighttpd       | 306    |
| namd      | 3      | grep      | 56     | xpdf-pdfinfo   | 387    |
| parest    | 1202   | gzip      | 73     | xpdf-pdfinfo   | 362    |
| povray    | 452    | mkdir     | 73     | xpdf-pdfimages | 368    |
| lbm       | 1      | rm        | 8      | xpdf-pdfimages | 363    |
| omnetpp   | 701    | sort      | 74     | xpdf-pdfimages | 365    |
| xalancbmk | 2473   | tar       | 58     | xpdf-pdfimages | 465    |
| x264      | 61     | uniq      | 36     |                |        |
| blender   | 2878   |           |        |                |        |
| deepsjeng | 27     |           |        |                |        |
| imagick   | 522    |           |        |                |        |
| leela     | 112    |           |        |                |        |
| nab       | 27     |           |        |                |        |
| xz        | 55     |           |        |                |        |

and maximum, respectively). This is the biggest jump (14.8%) for minimum reduction. The average is an improvement over SPEC but less than that for coreutils.

In comparison, BlankIt averages 1.047x runtime overhead and 98.9% ROP gadget reduction on nginx's libraries. As with SPEC, DECKERPLUS outperforms BlankIt at runtime but with less ROP gadget reduction. BlankIt copies needed library code into place before use and zeroes it out after use. This accounts for BlankIt's higher gadget reduction, and also explains why, despite only being used on libraries, BlankIt is slower than DECKERPLUS's page-mapping scheme. Neither Chisel nor Razor has been evaluated by their respective authors on nginx, making it difficult to compare DECKERPLUS against them.

*lighttpd*. We also evaluate lighttpd, a lightweight web server [30], using the same wrk-based security and timing experiments as nginx. This gives a second HTTP-serving workload under the same request structure, though lighttpd's simpler architecture produces stronger gadget reduction: its more direct call structure allows the request-serving path to collapse to a single executable page under inlining, whereas nginx's module dispatch and richer event model cause more pages to remain executable even under simple requests.

Unlike nginx, function cloning applies to lighttpd, cloning 306 functions (Table 3). Cloning requires statically visible direct call edges where the same function is called from sufficiently different contexts; nginx's module system dispatches through function pointers, so those edges are not visible to the cloning pass. lighttpd's simpler codebase exposes enough direct call structure for cloning to act on.

Table 2 shows that lighttpd has 2,138 raw gadgets in the unmodified binary. Baseline DECKERPLUS reduces the average gadget reduction to 79.8%. During the request-serving phase, 79 pages are simultaneously executable out of 307 pages in the binary as lighttpd dispatches through its connection state machine and request handlers, and the minimum reduction of 56.3% corresponds to this phase. Function cloning improves the average reduction to 97.5% by collapsing the request-serving phase from 79 pages to 3: page 71, which contains `http_request_headers_process`,

connection\_reset\_clone, and related request management functions; page 80, which contains URI validation; and page 90, which contains the event poll handler. Inlining improves the average reduction further to 99.3% and collapses the entire request-serving path to a single page containing http\_response\_prepare\_docroot, into which the full call chain has been inlined. The minimum reduction improves correspondingly, from 56.3% under baseline DECKERPLUS to 96.7% with cloning and 99.3% with inlining.

The timing results show that these security improvements do not introduce a corresponding runtime cost for the measured workload. Across the same duration and file-size tests used for nginx, all variants remain close to baseline throughput. The largest observed degradation is about 1.07x, and most measurements remain within a few percent of baseline. This suggests that the lighttpd workload exposes direct-call structure that is useful for cloning, but the cloned and inlined code does not add expensive runtime transitions on the request-serving path. In other words, lighttpd shows a case where the compiler extensions substantially improve both average and worst-case gadget reduction while preserving throughput under the tested wrk workloads.

*Redis.* Redis is a widely used in-memory data store [44]. We include it as an additional server workload with behavior distinct from HTTP request handling. For the security experiment, we start redis-server under DECKERPLUS's runtime logging and run a scripted workload that exercises strings, lists, sets, hashes, sorted sets, streams, transactions, Lua scripting, TTL/expiry operations, and a 200-key bulk SET/GET loop. This security run is separate from timing and is used to capture the available code pages for the gadget analysis reported in Table 2.

For timing, we use Redis's built-in redis-benchmark with 100,000 requests, 50 concurrent clients, and the SET, GET, LPUSH, LPOP, SADD, and HSET operations. Each variant is run for three trials and the results are averaged. Across these operations, throughput remains essentially unchanged: 64,194 requests/s for the baseline, 64,676 requests/s for DECKERPLUS, 64,514 requests/s with cloning, 64,469 requests/s with inlining. Thus, DECKERPLUS and its extensions do not introduce measurable throughput degradation for this Redis workload.

Table 2 shows that Redis has 21,505 raw gadgets in the unmodified binary. Baseline DECKERPLUS reduces the dynamically available gadget set by 96.3% on average. The high reduction reflects Redis's command-dispatch structure: each command activates a small, consistent set of pages comprising the jemalloc allocator pages, which are accessed on every command, plus one or two pages containing the specific command handler. During command execution, only 2–5 pages are simultaneously executable out of more than 600 pages. Pages corresponding to commands not exercised by the workload are never made executable. The minimum reduction corresponds to initialization, during which a larger portion of the binary is briefly executable before the event loop begins processing commands: in the baseline DECKERPLUS variant, more than 100 pages are simultaneously executable, with a maximum of 150, during initialization.

Function cloning improves the average reduction to 99.3%. It does so by reducing the executable page count during both initialization and per-command execution: with cloning, the maximum simultaneous page count falls to 37 and the average across all recorded execution points falls to 11.0 pages. Inlining improves the average reduction further to 99.6% by reducing the average simultaneous page count to 5.4 pages at the best threshold, although the maximum simultaneous page count increases to 76 pages. Both transformations reduce the initialization page count by eliminating call sites that would otherwise require those pages to be executable during startup. Together, the timing and security results show that Redis receives substantial gadget reduction with negligible runtime cost under this workload.

*xpdf.* Finally, we evaluate six command-line tools from xpdf [63]: pdftotext, pdfinfo, pdffonts, pdfdetach, pdfimages, and pdftops. These tools add a document-processing workload and

exercise parsing, metadata inspection, font analysis, attachment handling, image extraction, and PostScript conversion.

For the security experiment, each tool processes `PDF32000_2008.pdf`, a PDF specification document with 756 pages, under DECKERPLUS's runtime logging. Each xpdf tool performs one processing task on the same input document, and the resulting available-page sets are used for the gadget analysis in Table 2. For timing, we run each tool 20 times on the same PDF and measure wall-clock runtime with `/usr/bin/time`.

As shown in Table 2, the xpdf tools have between 7,736 and 9,922 raw gadgets in their unmodified binaries. Despite each binary linking the full xpdf rendering library, every tool's execution activates only a small fraction of its pages. `pdffonts` and `pdfdetach`, which enumerate font metadata and embedded attachments, respectively, activate at most two pages in any recorded execution context. One page contains the PDF graphics-state operators that initialize on every document open, while the second, when present, is application-specific. Baseline DECKERPLUS already achieves 99.3% average gadget reduction for `pdffonts` and 98.7% for `pdfdetach` because almost nothing in the rendering pipeline is reached. `pdftops` is the outlier: converting a PDF to PostScript requires traversing the full rendering path including color space handling, image encoding, and shading operations, so the available-page set grows to up to 54 pages, leaving a 96.9% average reduction, the weakest result among the six tools. Overall, baseline DECKERPLUS achieves 96.9–99.3% average total gadget reduction.

Function cloning achieves 97.7–99.4% average reduction across the six tools. Cloning benefits `pdftops`, where the average reduction improves from 96.9% to 99.4%. In this case, specializing the rendering functions reduces the available-page set from up to 54 pages to 24. However, cloning is counter-productive for `pdfinfo` and `pdfdetach`. The cloned variants of Gfx rendering operators each land on separate pages, and all of these pages are activated during document parsing, expanding the available-page set from 2–4 pages to 47–48. This is visible in the wide Min–Max spread for these tools, such as `pdfdetach` at 93.5–99.8%. Execution contexts that avoid the full parsing path see high reduction, while the document-parsing context activates all clone pages and sees lower reduction. The average result for `pdfinfo` falls from 98.9% under baseline DECKERPLUS to 97.7% under cloning.

Inlining achieves 99.7–99.9% average reduction across all six tools. It provides the largest improvement for `pdftops`. After DECKERPLUS's indirect call sinking converts the observed indirect calls in the rendering path to direct calls, the inliner can inline their target functions into the call sites. At runtime no deck activation is created for those inlined functions, so their pages never enter the available set. The peak 54-page available-page set collapses to at most three pages at the best threshold (700), improving average reduction from 96.9% to 99.8%. For the tools that were already narrow under baseline DECKERPLUS (`pdffonts`, `pdfdetach`, `pdfinfo`), inlining produces small but consistent further improvements, reaching 99.7–99.9%.

In the timing results, `pdftotext`, `pdfimages`, and `pdftops` provide the longest-running xpdf workloads. For these tools, baseline DECKERPLUS adds modest overhead, with slowdowns of 1.08×, 1.18×, and 1.09×, respectively, for an average slowdown of 1.12×. Function cloning is similar to baseline DECKERPLUS. Inlining at the selected threshold reduces overhead relative to baseline DECKERPLUS, with slowdowns of 1.06×, 1.14×, and 1.06×, respectively, for an average slowdown of 1.08×. The remaining tools, `pdfinfo`, `pdffonts`, and `pdfdetach`, complete in under 0.1s for the DECKERPLUS variants on this input, so small absolute timing differences are less meaningful for those workloads.

As with the server benchmarks, all gadget reduction measurements reflect the observed execution on the specific workload described above. DECKERPLUS's guarantee is always relative to the recorded execution. More diverse inputs or request patterns would cause additional pages to become executable and reduce the measured improvement accordingly.

## 6.4 Binary Size Growth

We measure the binary size increase over SPEC, coreutils, nginx, Redis, lighttpd, and the xpdf tools. With baseline DECKERPLUS, binary size increases by 2.9x across SPEC, 1.5x across coreutils, 1.8x for nginx, 1.29x for Redis, 1.53x for lighttpd, and 1.78x on average across the xpdf tools. In absolute terms, the modified binaries are 19.2MB for SPEC on average, 1.3MB for coreutils on average, 7.1MB for nginx, 29.8MB for Redis, 2.8MB for lighttpd, and 10.2MB on average for the xpdf tools. Thus, although DECKERPLUS increases binary size, the resulting binaries remain reasonably sized, and the performance measurements confirm that this growth does not adversely affect runtime for the measured workloads.

The difference between SPEC and coreutils is largely due to the number of disjoint sets. All binaries in coreutils have fewer than 200 sets, whereas the majority of SPEC benchmarks have 200 or more. This same intuition also helps explain the application results: larger programs and programs with more separated function sets tend to incur larger absolute binary sizes, while smaller server binaries such as lighttpd remain only a few megabytes even after instrumentation.

When we look at function cloning, the binary size increases to 3.4x on average across SPEC, 1.99x across coreutils, 1.60x for Redis, 2.40x for lighttpd, and 2.33x on average across the xpdf tools. We exclude nginx from the cloning-size comparison because no functions are cloned for nginx. The average absolute sizes of the cloned binaries are 21.6 MB for SPEC, 1.6 MB for coreutils, 37.1 MB for Redis, 4.4 MB for lighttpd, and 13.3 MB for the xpdf tools. Compared with baseline DECKERPLUS, cloning increases binary size by 20.4% for SPEC, 27.8% for coreutils, 24.2% for Redis, 57.4% for lighttpd, and 30.8% for the xpdf tools. These results are consistent with the clone counts reported in Table 3.

Several factors affect cloning-related growth, including the number of encompassed functions, the average function size, function alignment on pages, and the size of each function clone chain. A function clone chain includes the clones of an encompassed function and all functions reachable from that encompassed function. As a result, binary size can increase quickly when these chains contain many functions. Conversely, a large clone chain with small functions may increase size less than a smaller clone chain with larger functions.

When we look at inlining, binary size remains 2.9x across SPEC, but increases to 1.7x across coreutils, 2.0x for nginx, 1.7x for Redis, 1.7x for lighttpd, and 2.4x on average across the xpdf tools at the selected inlining thresholds. In absolute terms, the inlined binaries are 20 MB for SPEC on average, 1.4 MB for coreutils on average, 8.4 MB for nginx, 39.3 MB for Redis, 3.1 MB for lighttpd, and 13.7 MB on average for the xpdf tools. Inlining increases binary size because inlined functions make callers larger, and the original callee bodies cannot always be removed when they are still called elsewhere or may be reached through function pointers. Even so, the growth is generally less severe than function cloning.

For a representative subset of benchmarks, we also estimate the worst-case growth without custom linking. In this worst case, every function is assigned a page-aligned section, i.e., every disjoint set is a singleton. We lower-bound this estimate by ignoring weak function symbols in the baseline applications. For the evaluated subset, our custom linking script improves over the lower-bounded worst-case growth by 1.8x, 2.7x, and 1.3x for SPEC, coreutils, and nginx, respectively.

## 6.5 Breaking ROP Gadget Chains

Gadget reduction is a common metric in security-focused debloating works [22, 40, 41, 43, 52], but it is still difficult to draw certain conclusions. For example, even with 90% total gadget reduction, a significantly large enough binary could still have an enormous number of gadgets in absolute terms. Alternatively, 50% reduction may be good, provided that it removes a critical class of gadgets needed by any useful chain. Thus, gadget reduction is a useful yardstick, but we also need to investigate

Table 4. Execve-to-Shell ROP Chain Availability in Baseline Binaries and Whether DECKERPLUS Breaks the Complete Chain Across Observed Dynamic Page Sets

| App            | W-W-W | Args | Syscall | Baseline E2E | DECKERPLUS Breaks E2E |
|----------------|-------|------|---------|--------------|-----------------------|
| perlbench      | ✓     | ✓    | ✓       | ✓            | ✓                     |
| mcf            |       |      |         |              |                       |
| namd           |       |      | ✓       |              |                       |
| parest         | ✓     | ✓    | ✓       | ✓            | ✓                     |
| povray         | ✓     | ✓    | ✓       | ✓            | ✓                     |
| lbm            | ✓     |      |         |              |                       |
| omnetpp        | ✓     | ✓    | ✓       | ✓            | ✓                     |
| xalancbmk      | ✓     |      | ✓       |              |                       |
| x264           | ✓     | ✓    | ✓       | ✓            | ✓                     |
| deepsjeng      |       |      |         |              |                       |
| imagemagick    | ✓     | ✓    | ✓       | ✓            | ✓                     |
| leela          | ✓     |      |         |              |                       |
| nab            |       |      |         |              |                       |
| xz             | ✓     | ✓    |         |              |                       |
| bzip2          |       |      |         |              |                       |
| chown          | ✓     |      |         |              |                       |
| date           | ✓     |      | ✓       |              |                       |
| grep           |       |      | ✓       |              |                       |
| gzip           | ✓     |      |         |              |                       |
| mkdir          |       |      |         |              |                       |
| rm             | ✓     |      |         |              |                       |
| sort           | ✓     |      | ✓       |              |                       |
| tar            | ✓     |      |         |              |                       |
| uniq           | ✓     | ✓    | ✓       | ✓            | ✓                     |
| nginx          | ✓     | ✓    | ✓       | ✓            | ✓                     |
| redis          | ✓     | ✓    | ✓       | ✓            | ✓                     |
| lighttpd       | ✓     |      | ✓       |              |                       |
| xpdf-pdfdetach | ✓     | ✓    | ✓       | ✓            | ✓                     |
| xpdf-pdffonts  | ✓     | ✓    | ✓       | ✓            | ✓                     |
| xpdf-pdfimages | ✓     | ✓    | ✓       | ✓            | ✓                     |
| xpdf-pdfinfo   | ✓     | ✓    | ✓       | ✓            | ✓                     |
| xpdf-pdftops   | ✓     |      | ✓       |              |                       |
| xpdf-pdftotext | ✓     | ✓    | ✓       | ✓            | ✓                     |

whether an attack surface reduction technique actually removes attacks. We start by looking at DECKERPLUS's ability to break ROP chains.

Ropper [47] is an open-source tool that can automatically build gadget chains. It allows us to automatically test if we can successfully build a ROP gadget chain that spawns `/bin/sh` via an `execve` syscall. We use Ropper to identify which binaries from our experiments have this ROP gadget chain. Then we test every binary over all test inputs with DECKERPLUS and check every observed available page set for the ROP gadget chain. We run Ropper on SPEC, coreutils, nginx, Redis, lighttpd, and the xpdf tools.

Table 4 shows the prevalence of the ROP chain across all baseline applications' text sections. (blender's analysis times out, so it is not included.) Recall from Listing 1 that this ROP chain



requires a gadget to store values in memory, a sequence of gadgets to set up the syscall arguments, and a gadget to invoke the `execve` syscall. These correspond to Table 4's W-W-W, Args, and Syscall columns, respectively. (W-W-W stands for a write-what-where gadget.) If all of these components of the gadget chain are present for an application, then the end-to-end exploit exists (a checkmark in the "Baseline E2E" column). The final column reports whether DECKERPLUS removes the complete chain from every observed dynamic page set. Out of 33 baseline binaries/tools, 14 have the full ROP gadget chain, 14 have an incomplete chain, and 5 have no components of the chain.

**We analyze every observed available page set over all inputs across all applications with Ropper and find that DECKERPLUS does not allow the ROP chain under any set of dynamic decks.**

Importantly, this result does not depend on syscall instructions being absent from the binary. A single syscall instruction can be sufficient for an attacker if it is dynamically co-available with the gadgets needed to prepare memory, load the syscall number, set up the arguments, and transfer control to the syscall endpoint. We therefore analyze the chain components jointly for each available page set. Across all observed dynamic deck sets, no page set contains the complete combination of write-what-where gadgets, argument-staging gadgets, and syscall gadget needed for Ropper's end-to-end `execve` chain.

We analyze these chains and their broken counterparts under DECKERPLUS to validate the result and understand how DECKERPLUS breaks this ROP chain in practice. We show an analysis of `sort` to illustrate how a syscall opcode can appear even when it is not an intended syscall instruction, and why the corresponding page is not dynamically available in the observed DECKERPLUS executions.

The syscall opcode in `×86_64` is `0xF05`. Ropper identifies a syscall opcode in `sort` within a jump instruction. An objdump with 1 line of context is shown here:

```
4276ae: 41 b0 79      mov $0x79,%r8b
4276b1: e9 0f 05 00 00 jmpq 427bc5
4276b6: 80 fa 8a      cmp $0x7b,%dl
```

The jump target is `strftime_case_.254+0xf5`. The `e9` byte is the jump opcode, and `0f 05 00 00` is the addend to a base (which in little endian is `0x0000050F`). The base is calculated from the program counter (of the next instruction), which is `0x4276b6` in this case. Thus the jump target is `0x4276B6 + 0x050F`, which yields `0x427BC5`.

This code is part of the `strftime_case` function. This is a custom version of `strftime` from `glibc` that is built into `coreutils`. As such, it would not have been handled by load-time techniques like Piece-wise or library-only techniques like `BlankIt`.

In these experiments, this function's page is never mapped RX for `sort`; thus this syscall opcode is not available in any observed dynamic deck set.

## 6.6 Breaking JOP Gadget Chains

Next, we look at breaking JOP chains, which are more complex than ROP and also resilient against modern ROP defense mechanisms such as shadow stacks. A typical JOP chain consists of, among other details, a **dispatcher gadget (DG)** and **dispatch table (DT)** (refer to [4]). The DG is crucial for JOP chains, but it may be challenging to locate. A simple example of one is: `add 0x8 reg0; jmp *reg0`. It advances the DT address held in `reg0` then jumps to that gadget's address. When a gadget completes, it jumps back to the DG, and steps repeat.

**6.6.1 *nginx on Linux.*** We are not aware of any tools like Ropper that can automatically create JOP chains on Linux. Thus, we perform manual analysis for this chain-breaking study to provide a mix of quantitative and qualitative results. Two questions guide our analysis. First, can a JOP chain be formed that executes the notorious `execve` attack on `nginx` while running under DECKERPLUS?



Table 5. JOP Gadget Metrics Over All Available Page Sets  
Running Nginx on Linux (Baseline Shown in Average Column)

| Metric               | Min | Max | Avg        | Stdev |
|----------------------|-----|-----|------------|-------|
| all                  | 2   | 106 | 33.9 (357) | 27.9  |
| dispatcher           | 1   | 21  | 6.1 (28)   | 5.6   |
| uniq first op        | 1   | 28  | 12.1 (62)  | 7.4   |
| usable               | 0   | 74  | 25.7 (200) | 17.5  |
| uniq-first-op usable | 0   | 23  | 12.1 (50)  | 6.3   |

Second, is it possible to build some other functional JOP chain with the still available gadgets when running under DECKERPLUS?

Our first question is already answered by our previous ROP study, because, even when replacing the main portion of the ROP chain with JOP, we still fail to find the `0x0F05` syscall opcode needed for the final `execve` invocation. **Thus, the shell-spawning JOP chain is infeasible under DECKERPLUS for JOP chains, as well.**

To answer our second question, we run ROPGadget [45] on all dynamically captured available page sets running under DECKERPLUS for `nginx` and output all JOP gadgets. Then we scan for DGs. We find none that have the most useful, traditional form (`add immediate + indirect jump`). There are, however, length-2 gadgets of the form `add reg1 reg0; jmp *reg0`. This is sufficient, provided that `reg1` can be set to `0x8`, for example (which could be done with a properly constructed attack payload with `0x8` and `pop reg1`). Thus, we assume these gadgets are feasible for use as DGs.

Next, for each dynamically active page set, we analyze the available JOP gadgets that would be active if a given DG were chosen. In other words, we throw out gadgets that could potentially have side effects on the two registers that are being used for a chosen DG (`reg0` and `reg1` in our example); and we do this for all DGs available for a page set. This yields several results. First, it tells us if, at some point during execution, there was even a DG available for an attacker. Second, it tells us – after setting the DG registers – which JOP gadgets remain available for launching an attack.

We report 5 metrics for this analysis, shown in Table 5. These report the total number of gadgets, the number of DGs, and the number of unique first operands of those gadgets. Also, as described above in terms of choosing a DG, we report the total number of usable gadgets and the number of unique first operands that are usable once a DG and its registers are set.

The standard deviation for these metrics is instructive. For example, the total gadgets for a given page set averages 33.9 (low, indicating great defense), but the standard deviation is 27.9. When there are very few gadgets available, a quick inspection shows that there is nearly nothing that could conceivably be built. We must inspect the worst-case page sets, though, to understand if JOP chaining would still be difficult under these circumstances, even without a syscall gadget.

A good candidate is the available page set that contains the most number of unique first operands in its gadget list (after choosing the DG). Table 5 shows that the max observed value is 23. This occurs in our page set with ID=319. Note that we have not yet enforced more sophisticated restrictions, such as choosing a register to hold the address of the DG itself (which would be necessary for an actual attack), nor checking for stack pivot capability (often needed for an actual attack). In the former case, this must be taken into account. For page set 319, despite having 23 unique operands, these operands are not available or substantially abundant once we select the register that will hold the DG. For instance, there is a valid indirect write gadget that requires `rcx` to hold the DG address. However, there are no `pop` gadgets if we want to use it. Similarly, to benefit from the most `mov` gadgets (7 total), one must choose the `rsi` register to hold the DG's address, but there are again no `pop` gadgets that use `rsi` for jumping back to the DG.

In contrast, the baseline contains a much richer set of gadgets that could conceivably be used for more complex chains (Table 5, Avg column in parentheses). For example, there are 52 indirect write gadgets (instead of 3 for page set 319), and it is easy to identify pop gadgets that can use the same address for the DG. We ran another third-party tool on nginx to help validate this result. GSA [9] has some support for JOP gadgets and for ranking their “quality.” GSA’s total JOP score for the baseline is 789. For page set 319 it is 543.5 (31% reduction).

To summarize, an `execve`-invoking JOP chain is thwarted due to the missing `syscall` gadget. And, on closer inspection, a more novel, complex JOP chain is very difficult (no `write` or `mov` gadgets) or impossible to build (no usable gadgets), even when the dynamically available set of pages is most exposed. The lack of availability of a gadget breaks the necessary condition for constructing a JOP chain; we thus contend that DECKERPLUS is able to stop JOP chain-based attacks. In contrast, the baseline supports a substantially richer set of JOP gadgets; these provide the ability to launch an `execve` JOP chain; and they comprise a realistic set of *simultaneously present gadgets* for choosing registers used for the DG and DT, which is necessary to build chains beyond `execve`-invoking ones (e.g., the recent `shellcodeless` JOP technique [6]).

**6.6.2 *nginx* on Windows.** We conduct a JOP case study on Windows, as well. This is another critical platform, and it provides evidence that DECKERPLUS generalizes well beyond Linux. The only powerful open-source tool for JOP gadgets is also a Windows-only tool (JOP Rocket [5]), so we can leverage it for our study. JOP Rocket provides a user interface for interacting with an executable and its JOP gadgets. For example, the user can specify the set of registers available for building gadgets; they can scan for all gadgets that end in `jmp` (or `call`, or both); they can also instruct JOP Rocket to automatically build a complete JOP chain.

Our goals are similar to the previous subsection. We check whether JOP chains are broken by DECKERPLUS, and whether other novel JOP chains can still be built. We use the same workloads as before and capture the available page sets. Then we run JOP Rocket on every page set recorded at runtime and on a statically compiled `nginx` baseline.

In all cases, including the baseline, JOP Rocket is unable to automatically build a complete JOP chain. This does not imply that there are no JOP chains – only that JOP Rocket could not automatically construct one. (There are multiple reasons why a chain may not be found. The JOP Rocket paper at Black Hat is useful for learning more [5].)

It is for this reason that fine-grained metrics are extremely useful in gauging the effectiveness of DECKERPLUS on debloating JOP gadgets. JOP Rocket makes this task much easier than on Linux, so we provide a more interesting set of reduction metrics in Table 6.

As with Linux, this list of metrics shows how DECKERPLUS can make JOP-based attacks much more difficult. For example, the maximum reduction for DECKERPLUS under the `nginx` workloads is 100% for each case. In the average case, the results are similar to the overall numbers presented previously. For example, we report `call` gadgets here as well, showing that on average 78% are eliminated from the available page sets. Length-2 gadgets (usually desirable to attackers because of the limited side effects) are reduced by 47%. In one case, DECKERPLUS succeeds in eliminating an entire class of gadgets under the given workload, namely the `mov-immediate` gadgets present in the baseline. Stack pivot gadgets are crucial for many JOP chains (e.g., those that need to set up arguments for a function call). DECKERPLUS eliminates 100% of these in the best case and reduces this by 52% on average. This can be sufficient for thwarting an attack if the attacker cannot properly pivot the stack during the chain. One other interesting metric is the number of 2-gadget dispatchers, which are reduced by 60% on average. These dispatchers are introduced in [5], and JOP Rocket can find these gadgets for manually building chains from them; however, there are 60% fewer such gadgets on average due to DECKERPLUS.

Table 6. JOP Reduction on Windows Nginx (Higher is Better)

| Metric                    | Min  | Max | Avg  |
|---------------------------|------|-----|------|
| call gadgets              | 0.65 | 1   | 0.78 |
| len-2 gadgets             | 0.05 | 1   | 0.47 |
| len-2 mov gadgets         | 0.78 | 1   | 0.81 |
| max len-3 add ops         | 0.88 | 1   | 0.99 |
| stack pivot gadgets       | 0.5  | 1   | 0.52 |
| 2-gadget dispatchers      | 0.5  | 1   | 0.6  |
| uniq addends for add op   | 0.38 | 1   | 0.54 |
| uniq dec ops              | 0.96 | 1   | 0.97 |
| uniq gadgets              | 0.25 | 1   | 0.6  |
| uniq inc ops              | 0.42 | 1   | 0.76 |
| uniq jump regs for add op | 0.5  | 1   | 0.77 |
| uniq len-2 addends to esp | 0    | 1   | 0.99 |
| uniq mov imm insts        | 1    | 1   | 1    |
| uniq sub ops              | 0.26 | 1   | 0.46 |

## 7 Related Work

Control-flow integrity (CFI) restricts forward and backward control-flow transfers to permitted edges in the control-flow graph or callgraph [1, 20]. Numerous systems have extended CFI to different execution environments and threat models [23, 38, 58, 59], but practical limitations remain. Prior work has demonstrated bypasses and identified weaknesses caused by imprecise analysis, invalid runtime assumptions, unprotected corner cases, compiler optimizations, implementation errors, and unintended targets [11, 21, 29]. For example,  $\mu$ CFI cannot prevent code-pointer reuse and VTable attacks [23], while OS-CFI does not protect some tail calls that are optimized into indirect calls [24]. Industry defenses such as Windows Control Flow Guard and Intel CET also remain vulnerable to code-reuse attacks [5, 32, 51]. Attacks may exploit counterfeit objects, data-only corruption, race conditions, or mechanisms such as NTContinue [48, 54], and software-based CFG may be bypassed when applications or libraries lack complete protection [65].

ROPecker [12] is a runtime ROP defense that also dynamically changes page permissions. It uses a sliding execution window and marks code outside the current region non-executable so that an ongoing gadget chain triggers detection. Although both systems manipulate page permissions, their goals differ. ROPecker detects and halts an attack in progress, whereas DECKERPLUS reduces the gadgets simultaneously available throughout normal execution so that a complete chain is less likely to become exposed. Control pointer integrity (CPI) instead protects code pointers [26, 27], although attacks such as control-flow bending may remain outside its scope [11]. CFI and CPI seek to preserve valid control flow, while DECKERPLUS and other attack-surface-reduction systems limit the code available after control-flow corruption. These defenses are complementary: a successful CFI bypass may still fail when required gadgets are unavailable, while CFI may prevent an attacker from reaching a gadget chain that remains in a reduced program.

Several systems reduce attack surface according to application features, configurations, or observed behavior. Slimium combines static, dynamic, and heuristic analysis to remove Chromium features [42]. Koo et al. remove configuration-specific code and report 77% reduction for nginx [25], while Trimmer uses a user-provided configuration to specialize programs [52]. Soto-Valero et al. use coverage information to reduce Java library bytecode [53]. These techniques depend on explicit features, configurations, or observed coverage, whereas DECKERPLUS requires no user-supplied workload or specification. ELFbac [3] also restricts access to regions of ELF binaries, but it relies

on user-defined policies over sections and metadata-level regions. DECKERPLUS instead derives temporal function-level policies automatically, provides finer-grained control, and is not tied to ELF section metadata.

Recent work has explored input-driven and detection-based defenses against code-reuse attacks. Wang et al. propose Picup [61], an online library-debloating system that uses a convolutional neural network to predict the functions required for each input. Underprediction may remove required functionality, while overprediction leaves additional code exposed. In contrast, DECKERPLUS conservatively derives decks from the callgraph, requires no learned model or input-specific prediction, and operates on the whole application. An orthogonal approach uses hardware performance counters and machine learning to detect attacks during execution. Welearegai et al. demonstrate this approach on ARM and report 75% online detection accuracy with modest runtime overhead [62]. Detection and attack-surface reduction are complementary because DECKERPLUS reduces the executable surface available to an attacker, while a runtime detector may identify attacks that remain possible within an available deck.

Debloating has also been studied for performance and code-size reduction rather than security. Prior work examines the causes and utility of software bloat and applies debloating to applications and containers [33, 34, 64], while earlier compiler and linker techniques remove unnecessary code to reduce binary size [18, 35]. In contrast, DECKERPLUS provides automated, sound, whole-application attack surface reduction with low runtime overhead. It is a software-only technique that complements existing hardware, control-flow, and attack-detection defenses.

## 8 Conclusion

We present DECKERPLUS, an attack surface reduction technique that works on full programs and can enable may-use code on-demand. It achieves strong gadget reduction results without compromising soundness or requiring training, user inputs, or specifications. Across SPEC CPU 2017, GNU coreutils, and the standalone application workloads, baseline DECKERPLUS achieves average total gadget reductions of 70.3%, 88.5%, and 89.0%, respectively. Function cloning improves these averages to 75.5%, 89.6%, and 94.2%, and inlining improves them further to 77.6%, 94.7%, and 97.8%.

In our performance experiments, the average runtime slowdown on SPEC is 6.5% and overhead is negligible for GNU coreutils. For the server workloads, nginx averages 1.062x slowdown with baseline DECKERPLUS and 1.011x with inlining, while Redis throughput is essentially unchanged and lighttpd remains close to baseline throughput. The xpdf tools also show modest overhead on the longest-running document processing workloads.

In an additional study over these applications, we show that for all test inputs, DECKERPLUS eliminates the presence of a ROP chain that spawns a shell via `execve`. We show similar results in an nginx JOP study for both Linux and Windows. Based on these results and the generality of the approach, we find DECKERPLUS and its extensions to be a promising technique for attack surface reduction in practice.

## References

- [1] Martín Abadi, Mihai Budiu, Úlfar Erlingsson, and Jay Ligatti. 2005. Control-flow integrity. In *Proceedings of the 12th ACM Conference on Computer and Communications Security, CCS 2005*, Vijay Atluri, Catherine A. Meadows, and Ari Juels (Eds.). ACM, 340–353. DOI: <https://doi.org/10.1145/1102120.1102165>
- [2] S. Andersen and V. Abella. 2004. Data Execution Prevention: Changes to Functionality in Microsoft Windows XP Service Pack 2, Part 3: Memory Protection Technologies. Retrieved October 10, 2021 from <http://technet.microsoft.com/en-us/library/bb457155.aspx>
- [3] Julian Bangert, Sergey Bratus, Rebecca Shapiro, Michael E. Locasto, Jason Reeves, Sean W. Smith, and Anna Shubina. 2013. *ELFbac: Using the Loader Format for Intent-Level Semantics and Fine-Grained Protection*. Technical Report TR2013-727. Dartmouth University. Retrieved from [https://digitalcommons.dartmouth.edu/cs\\_tr/345](https://digitalcommons.dartmouth.edu/cs_tr/345)

- [4] Tyler K. Bletsch, Xuxian Jiang, Vincent W. Freeh, and Zhenkai Liang. 2011. Jump-oriented programming: A new class of code-reuse attack. In *Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security, ASIACCS 2011*, Bruce S. N. Cheung, Lucas Chi Kwong Hui, Ravi S. Sandhu, and Duncan S. Wong (Eds.). ACM, 30–40. DOI : <https://doi.org/10.1145/1966913.1966919>
- [5] Bramwell Brizendine and Austin Babcock. 2021. Pre-built JOP Chains with the JOP ROCKET: Bypassing DEP without ROP. Retrieved June 06, 2022 from <https://i.blackhat.com/asia-21/Thursday-Handouts/as-21-Brizendine-Babcock-Prebuilt-Jop-Chains-With-The-Jop-Rocket-wp.pdf>
- [6] Bramwell Brizendine and Austin Babcock. 2021. Shellcodeless JOP: Advanced Code-Reuse Attacks with Jump-Oriented Programming. Retrieved June 23, 2022 from [https://files.athack.com/files/2021-12/AtHack-Advanced%20Code-Reuse%20Attacks%20Whitepaper\\_0.pdf?VersionId=D.HV08jcDUeclPb4Fclic2jpOuK07GR8](https://files.athack.com/files/2021-12/AtHack-Advanced%20Code-Reuse%20Attacks%20Whitepaper_0.pdf?VersionId=D.HV08jcDUeclPb4Fclic2jpOuK07GR8)
- [7] Michael D. Brown, Adam Meily, Brian Fairservice, Akshay Sood, Jonathan Dorn, Eric Kilmer, and Ronald Eytchison. 2024. A broad comparative evaluation of software debloating tools. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 3927–3943. Retrieved from <https://www.usenix.org/conference/usenixsecurity24/presentation/brown>
- [8] Michael D. Brown and Santosh Pande. 2019. Is less really more? Towards better metrics for measuring security improvements realized through software debloating. In *Proceedings of the 12th USENIX Workshop on Cyber Security Experimentation and Test, CSET 2019*, Rob Jansen and Peter A. H. Peterson (Eds.). USENIX Association. Retrieved from <https://www.usenix.org/conference/cset19/presentation/brown>
- [9] Michael D. Brown, Matthew Pruett, Robert Bigelow, Girish Mururu, and Santosh Pande. 2021. Not so fast: Understanding and mitigating negative impacts of compiler optimizations on code reuse gadget sets. *Proc. ACM Program. Lang.* 5, OOPSLA (2021), 1–30. DOI : <https://doi.org/10.1145/3485531>
- [10] CallGraph.h 2020. CallGraph.h. Retrieved November 9, 2022 from <https://github.com/llvm/llvm-project/blob/llvmorg-11.0.0/llvm/include/llvm/Analysis/CallGraph.h>
- [11] Nicolas Carlini, Antonio Barresi, Mathias Payer, David Wagner, and Thomas R. Gross. 2015. Control-flow bending: On the effectiveness of control-flow integrity. In *Proceedings of the 24th USENIX Conference on Security Symposium (SEC’15)*. USENIX Association, Berkeley, CA, USA, 161–176. Retrieved from <http://dl.acm.org/citation.cfm?id=2831143.2831154>
- [12] Yueqiang Cheng, Zongwei Zhou, Miao Yu, Xuhua Ding, and Robert H. Deng. 2014. ROPecker: A generic and practical approach for defending against ROP attacks. In *Proceedings 2014 Network and Distributed System Security Symposium*. Internet Society, San Diego, CA. DOI : <https://doi.org/10.14722/ndss.2014.23156>
- [13] Catalin Cimpanu. 2021. Chrome: 70 percent of all security bugs are memory safety issues. Retrieved July 27, 2021 from <https://www.zdnet.com/article/chrome-70-of-all-security-bugs-are-memory-safety-issues/>
- [14] Catalin Cimpanu. 2021. Microsoft: 70 percent of all security bugs are memory safety issues. Retrieved July 27, 2021 from <https://www.zdnet.com/article/microsoft-70-percent-of-all-security-bugs-are-memory-safety-issues/>
- [15] coreutils 2021. Coreutils - GNU core utilities. Retrieved July 27, 2021 from <https://www.gnu.org/software/coreutils/>
- [16] Standard Performance Evaluation Corporation. 2021. SPEC CPU 2017. Retrieved July 27, 2021 from <https://www.spec.org/cpu2017/>
- [17] cxreel. 2021. Razor. Retrieved September 30, 2021 from <https://github.com/cxreel/razor>
- [18] Saumya K. Debray, William Evans, Robert Muth, and Bjorn De Sutter. 2000. Compiler techniques for code compaction. *ACM Trans. Program. Lang. Syst.* 22, 2 (March 2000), 378–415. DOI : <https://doi.org/10.1145/349214.349233>
- [19] Tyler Durden. 2002. Bypassing PaX ASLR protection. Retrieved September 14, 2021 from <http://phrack.org/issues/59/9.html>
- [20] Úlfar Erlingsson, Martín Abadi, Michael Vrabie, Mihai Budiu, and George C. Necula. 2006. XFI: Software guards for system address spaces. In *Proceedings of the 7th Symposium on Operating Systems Design and Implementation (OSDI’06)*, Brian N. Bershad and Jeffrey C. Mogul (Eds.). USENIX Association, 75–88. Retrieved from <http://www.usenix.org/events/osdi06/tech/erlingsson.html>
- [21] Isaac Evans, Fan Long, Ulziibayar Otgonbaatar, Howard Shrobe, Martin Rinard, Hamed Okhravi, and Stelios Sidiroglou-Douskos. 2015. Control jujutsu: On the weaknesses of fine-grained control flow integrity. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS’15)*. ACM, New York, NY, USA, 901–913. DOI : <https://doi.org/10.1145/2810103.2813646>
- [22] Kihong Heo, Woosuk Lee, Pardis Pashakhanloo, and Mayur Naik. 2018. Effective program debloating via reinforcement learning. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS’18)*. ACM, New York, NY, USA, 380–394. DOI : <https://doi.org/10.1145/3243734.3243838>
- [23] Hong Hu, Chenxiong Qian, Carter Yagemann, Simon Pak Ho Chung, William R. Harris, Taesoo Kim, and Wenke Lee. 2018. Enforcing unique code target property for control-flow integrity. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS’18)*. ACM, New York, NY, USA, 1470–1486. DOI : <https://doi.org/10.1145/3243734.3243797>



- [24] Mustakimur Khandaker, Wenqing Liu, Abu Naser, Zhi Wang, and Jie Yang. 2019. Origin-sensitive control flow integrity. In *Proceedings of the 28th USENIX Security Symposium, USENIX Security 2019*, Nadia Heninger and Patrick Traynor (Eds.). USENIX Association, 195–211. Retrieved from <https://www.usenix.org/conference/usenixsecurity19/presentation/khandaker>
- [25] Hyungjoon Koo, Seyedhamed Ghavamnia, and Michalis Polychronakis. 2019. Configuration-driven software debloating. In *Proceedings of the 12th European Workshop on Systems Security, EuroSec@EuroSys 2019*. ACM, 9:1–9:6. DOI: <https://doi.org/10.1145/3301417.3312501>
- [26] Volodymyr Kuznetsov, László Szekeres, Mathias Payer, George Candea, R. Sekar, and Dawn Song. 2014. Code-pointer integrity. In *Proceedings of the 11th USENIX Conference on Operating Systems Design and Implementation (OSDI'14)*. USENIX Association, Berkeley, CA, USA, 147–163. Retrieved from <http://dl.acm.org/citation.cfm?id=2685048.2685061>
- [27] Volodymyr Kuznetsov, Laszlo Szekeres, Mathias Payer, George Candea, and Dawn Song. 2015. Poster: Getting the point(er): On the feasibility of attacks on code-pointer integrity. In *36th IEEE Symposium on Security and Privacy*.
- [28] Chris Lattner and Vikram S. Adve. 2004. LLVM: A compilation framework for lifelong program analysis & transformation. In *Proceedings of the 2nd IEEE/ACM International Symposium on Code Generation and Optimization (CGO 2004)*. IEEE Computer Society, 75–88. DOI: <https://doi.org/10.1109/CGO.2004.1281665>
- [29] Yuan Li, Mingzhe Wang, Chao Zhang, Xingman Chen, Songtao Yang, and Ying Liu. 2020. Finding cracks in shields: On the security of control flow integrity mechanisms. In *CCS'20: 2020 ACM SIGSAC Conference on Computer and Communications Security*, Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna (Eds.). ACM, 1821–1835. DOI: <https://doi.org/10.1145/3372297.3417867>
- [30] lighttpd 2026. lighttpd. Retrieved April 28, 2026 from <https://github.com/lighttpd/lighttpd1.4>
- [31] Nicholas D. Matsakis and Felix S. Klock II. 2014. The rust language. In *Proceedings of the 2014 ACM Sigada Annual Conference on High Integrity Language Technology, HILT 2014*, Michael Feldman and S. Tucker Taft (Eds.). ACM, 103–104. DOI: <https://doi.org/10.1145/2663171.2663188>
- [32] Microsoft. 2022. Control Flow Guard for platform security. Retrieved October 21, 2022 from <https://learn.microsoft.com/en-us/windows/win32/secbp/control-flow-guard>
- [33] Nick Mitchell, Edith Schonberg, and Gary Sevitsky. 2009. Making sense of large heaps. In *Proceedings of the 23rd European Conference on ECOOP 2009—Object-Oriented Programming*, Sophia Drossopoulou (Ed.). Lecture Notes in Computer Science, Vol. 5653, Springer, 77–97. DOI: [https://doi.org/10.1007/978-3-642-03013-0\\_5](https://doi.org/10.1007/978-3-642-03013-0_5)
- [34] Nick Mitchell and Gary Sevitsky. 2007. The causes of bloat, the limits of health. In *Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2007, October 21–25, 2007*, Richard P. Gabriel, David F. Bacon, Cristina Videira Lopes, and Guy L. Steele Jr. (Eds.). ACM, 245–260. DOI: <https://doi.org/10.1145/1297027.1297046>
- [35] Robert Muth, Saumya K. Debray, Scott Watterson, and Koen De Bosschere. 2001. Alto: A link-time optimizer for the compaq alpha. *Softw. Pract. Exper.* 31, 1 (Jan. 2001), 67–101. DOI: [https://doi.org/10.1002/1097-024X\(200101\)31:1<67::AID-SPE357>3.0.CO;2-A](https://doi.org/10.1002/1097-024X(200101)31:1<67::AID-SPE357>3.0.CO;2-A)
- [36] Nergal. 2001. The Advanced Return-into-lib(c) Exploits: PaX Case Study. Phrack Magazine. Retrieved October 10, 2021 from <http://phrack.org/issues/58/4.html>
- [37] nginx 2019. nginx. Retrieved October 10, 2021 from <https://nginx.org/>
- [38] Ben Niu and Gang Tan. 2014. Modular control-flow integrity. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI'14*, Michael F. P. O'Boyle and Keshav Pingali (Eds.). ACM, 577–587. DOI: <https://doi.org/10.1145/2594291.2594295>
- [39] PAX ASLR 2003. PaX Address Space Layout Randomization. Retrieved October 10, 2021 from <https://pax.grsecurity.net/docs/aslr.txt>
- [40] Chris Porter, Girish Mururu, Prithayan Barua, and Santosh Pande. 2020. BlankIt library debloating: Getting what you want instead of cutting what you don't. In *Proceedings of the 41st ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI 2020*, Alastair F. Donaldson and Emina Torlak (Eds.). ACM, 164–180. DOI: <https://doi.org/10.1145/3385412.3386017>
- [41] Chenxiong Qian, Hong Hu, Mansour Alharthi, Simon Pak Ho Chung, Taesoo Kim, and Wenke Lee. 2019. RAZOR: A framework for post-deployment software debloating. In *Proceedings of the 28th USENIX Security Symposium, USENIX Security 2019*, Nadia Heninger and Patrick Traynor (Eds.). USENIX Association, 1733–1750. Retrieved from <https://www.usenix.org/conference/usenixsecurity19/presentation/qian>
- [42] Chenxiong Qian, Hyungjoon Koo, ChangSeok Oh, Taesoo Kim, and Wenke Lee. 2020. Slimium: Debloating the chromium browser with feature subsetting. In *CCS'20: 2020 ACM SIGSAC Conference on Computer and Communications Security*, Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna (Eds.). ACM, 461–476. DOI: <https://doi.org/10.1145/3372297.3417866>
- [43] Anh Quach, Aravind Prakash, and Lok Yan. 2018. Debloating software through piece-wise compilation and loading. In *Proceedings of the 27th USENIX Security Symposium (USENIX Security 18)*. USENIX Association, Baltimore, MD, 869–886. Retrieved from <https://www.usenix.org/conference/usenixsecurity18/presentation/quach>

- [44] redis 2026. Redis. Retrieved April 28, 2026 from <https://github.com/redis/redis>
- [45] ROPgadget 2018. ROPgadget v5.4. Retrieved October 10, 2021 from <https://github.com/JonathanSalwan/ROPgadget>
- [46] AliAkbar Sadeghi, Salman Niksefat, and Maryam Rostamipour. 2018. Pure-call oriented programming (PCOP): Chaining the gadgets using call instructions. *J. Comput. Virol. Hacking Tech.* 14, 2 (2018), 139–156. DOI: <https://doi.org/10.1007/s11416-017-0299-1>
- [47] Sascha Schirra. 2021. Ropper. Retrieved September 10, 2021 from <https://github.com/sashes/ropper>
- [48] Felix Schuster, Thomas Tendyck, Christopher Liebchen, Lucas Davi, Ahmad-Reza Sadeghi, and Thorsten Holz. 2015. Counterfeit object-oriented programming: On the difficulty of preventing code reuse attacks in C++ applications. In *Proceedings of the 2015 IEEE Symposium on Security and Privacy, SP 2015*. IEEE Computer Society, 745–762. DOI: <https://doi.org/10.1109/SP.2015.51>
- [49] Hovav Shacham. 2007. The geometry of innocent flesh on the bone: return-into-libc without function calls (on the x86). In *Proceedings of the 2007 ACM Conference on Computer and Communications Security, CCS 2007*, Peng Ning, Sabrina De Capitani di Vimercati, and Paul F. Syverson (Eds.). ACM, 552–561. DOI: <https://doi.org/10.1145/1315245.1315313>
- [50] Hovav Shacham, Matthew Page, Ben Pfaff, Eu-Jin Goh, Nagendra Modadugu, and Dan Boneh. 2004. On the effectiveness of address-space randomization. In *Proceedings of the 11th ACM Conference on Computer and Communications Security, CCS 2004*, Vijayalakshmi Atluri, Birgit Pfizmann, and Patrick D. McDaniel (Eds.). ACM, 298–307. DOI: <https://doi.org/10.1145/1030083.1030124>
- [51] Vedvyas Shanbhogue, Deepak Gupta, and Ravi Sahita. 2019. Security analysis of processor instruction set architecture for enforcing control-flow integrity. In *Proceedings of the 8th International Workshop on Hardware and Architectural Support for Security and Privacy (HASP'19)*. ACM, New York, NY, USA, Article 8, 11 pages. DOI: <https://doi.org/10.1145/3337167.3337175>
- [52] Hashim Sharif, Muhammad Abubakar, Ashish Gehani, and Fareed Zaffar. 2018. TRIMMER: Application specialization for code debloating. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, ASE 2018*, Marianne Huchard, Christian Kästner, and Gordon Fraser (Eds.). ACM, 329–339. DOI: <https://doi.org/10.1145/3238147.3238160>
- [53] César Soto-Valero, Thomas Durieux, Nicolas Harrand, and Benoit Baudry. 2023. Coverage-based debloating for Java bytecode. *ACM Trans. Softw. Eng. Methodol.* 32, 2, Article 38 (2023), 1–34. DOI: <https://doi.org/10.1145/3546948>
- [54] Bing Sun, Jin Liu, and Chong Xu. 2019. How to Survive the Hardware-assisted Control-flow Integrity Enforcement. Retrieved June 06, 2022 from <https://i.blackhat.com/asia-19/Thu-March-28/bh-asia-Sun-How-to-Survive-the-Hardware-Assisted-Control-Flow-Integrity-Enforcement.pdf>
- [55] Vijay Sundaresan, Laurie Hendren, Chrislain Razafimahefa, Raja Vallée-Rai, Patrick Lam, Etienne Gagnon, and Charles Godin. 2000. Practical virtual method call resolution for Java. In *Proceedings of the 15th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA'00)*. ACM, New York, NY, USA, 264–280. DOI: <https://doi.org/10.1145/353171.353189>
- [56] Minh Tran, Mark Etheridge, Tyler Bletsch, Xuxian Jiang, Vincent Freeh, and Peng Ning. 2011. On the expressiveness of return-into-libc attacks. In *Recent Advances in Intrusion Detection*, Robin Sommer, Davide Balzarotti, and Gregor Maier (Eds.). Springer Berlin Heidelberg, Berlin, 121–141.
- [57] Mircea Trofin, Yundi Qian, Eugene Brevdo, Zinan Lin, Krzysztof Choromanski, and David Li. 2021. Mlgo: A machine learning guided compiler optimizations framework. arXiv:2101.04808. Retrieved from <https://arxiv.org/abs/2101.04808>
- [58] Victor van der Veen, Dennis Andriess, Enes Göktas, Ben Gras, Lionel Sambuc, Asia Slowinska, Herbert Bos, and Cristiano Giuffrida. 2015. Practical context-sensitive CFI. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, Indrajit Ray, Ninghui Li, and Christopher Kruegel (Eds.). ACM, 927–940. DOI: <https://doi.org/10.1145/2810103.2813673>
- [59] Victor van der Veen, Enes Göktas, Moritz Contag, Andre Pawoloski, Xi Chen, Sanjay Rawat, Herbert Bos, Thorsten Holz, Elias Athanasopoulos, and Cristiano Giuffrida. 2016. A tough call: Mitigating advanced code-reuse attacks at the binary level. In *Proceedings of the IEEE Symposium on Security and Privacy, SP 2016*. IEEE Computer Society, 934–953. DOI: <https://doi.org/10.1109/SP.2016.60>
- [60] W3Techs. 2021. Usage statistics of web servers. Retrieved October 10, 2021 from [https://w3techs.com/technologies/overview/web\\_server](https://w3techs.com/technologies/overview/web_server)
- [61] Xiaoke Wang, Tao Hui, Lei Zhao, and Yueqiang Cheng. 2023. Input-driven dynamic program debloating for code-reuse attack mitigation. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, San Francisco CA USA, 934–946. DOI: <https://doi.org/10.1145/3611643.3616274>
- [62] Gebrehiwet B. Welearegai, Chenpo Hu, and Christian Hammer. 2023. Detecting and preventing ROP attacks using machine learning on arm. In *Proceedings of the 2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, Torino, Italy, 667–677. DOI: <https://doi.org/10.1109/COMPSAC57700.2023.00092>
- [63] xpdf 2017. Xpdf 4.00. Retrieved April 28, 2026 from <https://github.com/kermitt2/xpdf-4.00>



- [64] Guoqing Xu and Atanas Rountev. 2010. Detecting inefficiently-used containers to avoid bloat. In *Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI'10)*. ACM, New York, NY, USA, 160–173. DOI : <https://doi.org/10.1145/1806596.1806616>
- [65] Zhang Yunha. 2015. Bypass control flow guard comprehensively. Black Hat USA 2015. Retrieved August 13, 2026 from <https://www.blackhat.com/docs/us-15/materials/us-15-Zhang-Bypass-Control-Flow-Guard-Comprehensively-wp.pdf>

Received 20 October 2025; revised 12 May 2026; accepted 11 July 2026